



操作系统原理

Operating Systems Principles

陈鹏飞
计算机学院



第四讲 — 进程



概述

- ❖ 进程概念
- ❖ 进程调度
- ❖ 进程上的操作
- ❖ 进程间通信
- ❖ IPC在共享存储系统中的应用
- ❖ 消息传递系统中的IPC
- ❖ IPC系统示例
- ❖ 客户机-服务器系统中的通信



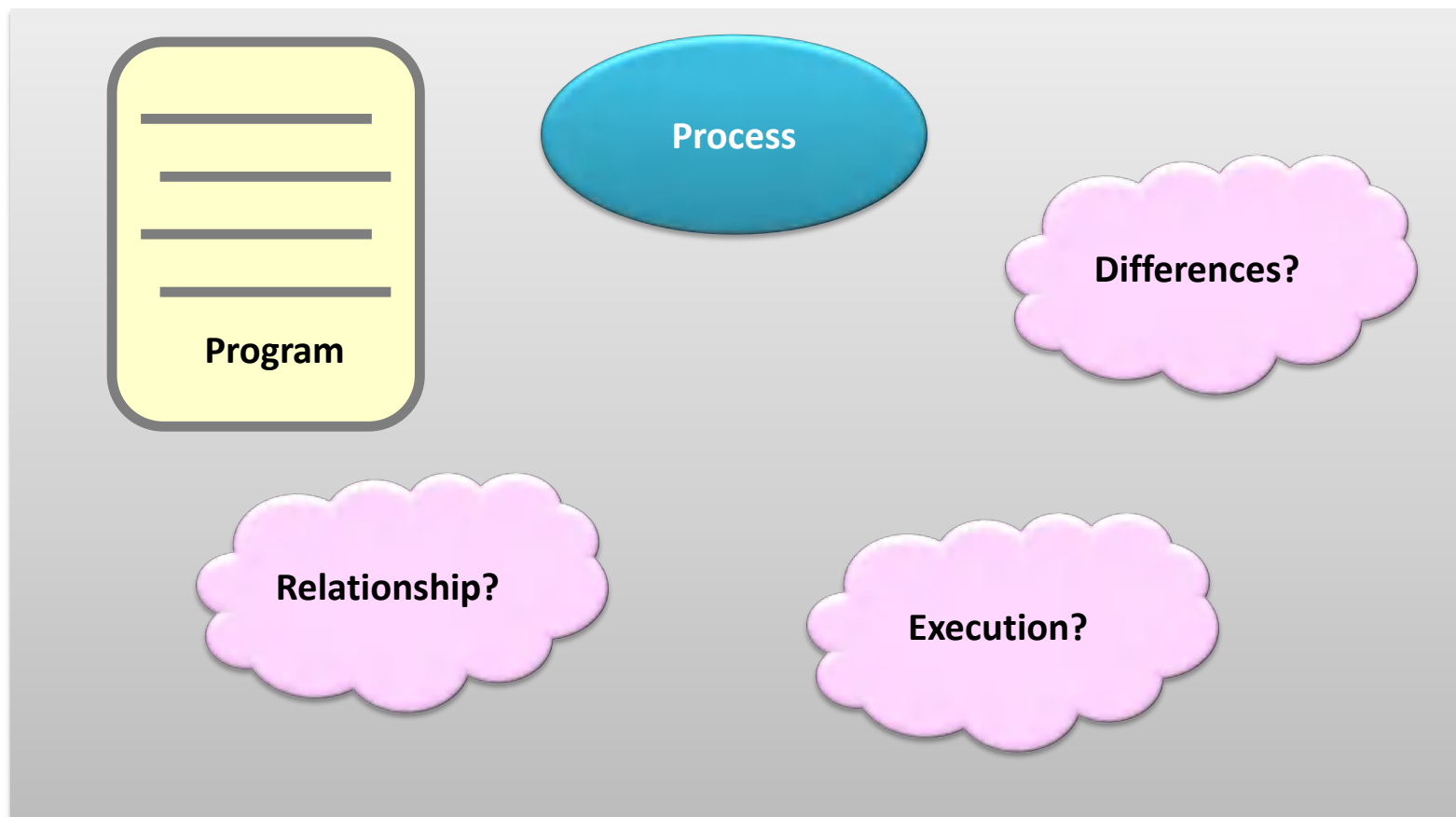
目标

- ❖ 确定进程的各个组件，并说明如何在操作系统中表示和调度这些组件。
- ❖ 描述如何在操作系统中创建和终止进程，包括使用执行这些操作的适当系统调用开发程序。
- ❖ 描述并对比使用共享内存和消息传递的进程间通信。
- ❖ 设计使用管道和POSIX共享内存执行进程间通信的程序。
- ❖ 描述使用套接字和远程过程调用的客户机-服务器通信。
- ❖ 设计与Linux操作系统交互的内核模块。



进程是什么？

Informally, a process is a program in execution.





什么是程序

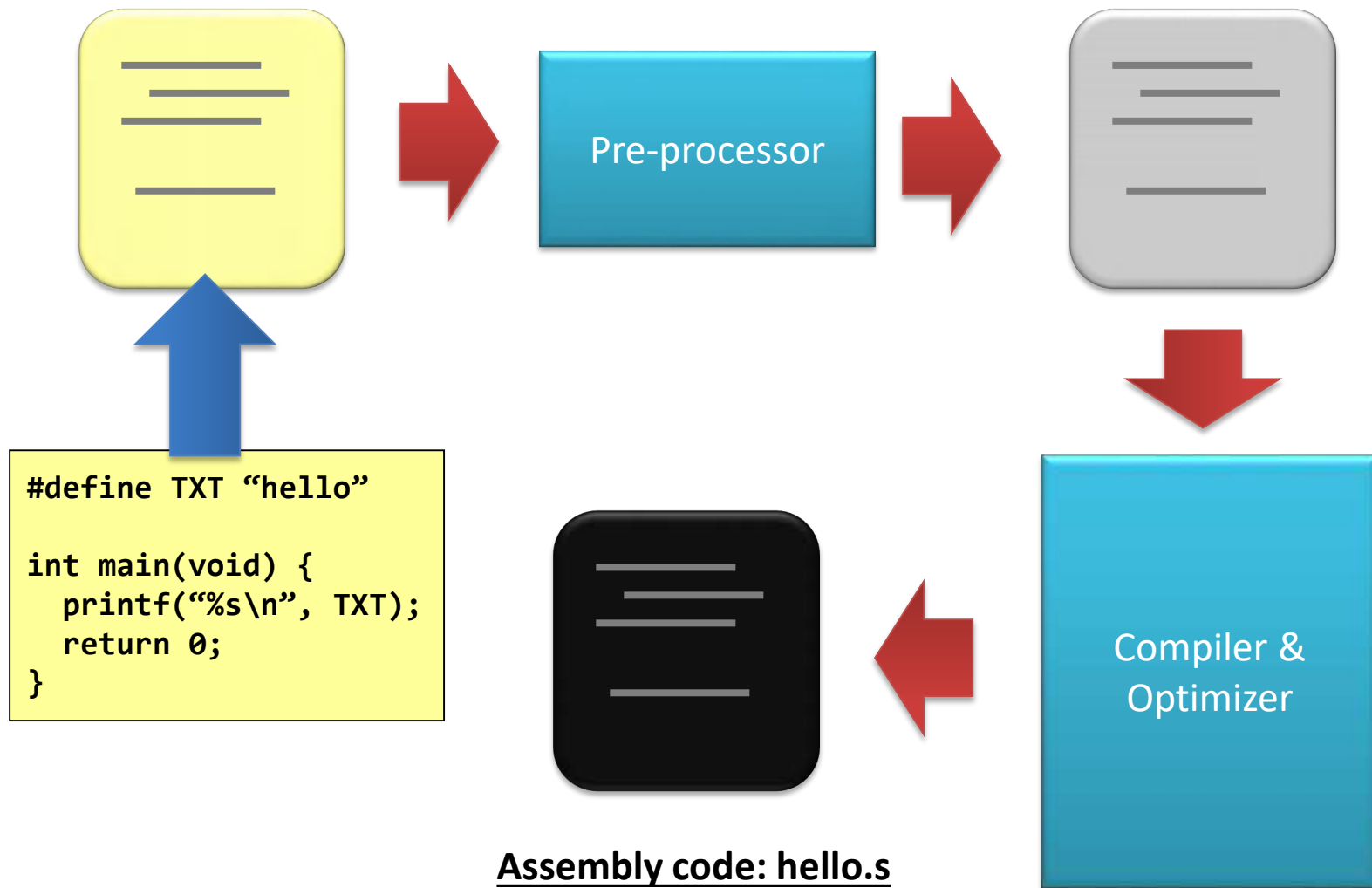
- What is a program?
 - A program is a just **a piece of code**.
- But, which code do you mean?
 - High-level language code: C or C++?
 - Low-level language code: assembly code?
 - Not-yet an executable: object code?
 - Executable: machine code?



创建一个程序的流程

C code: hello.c

Expanded C code: hello.c

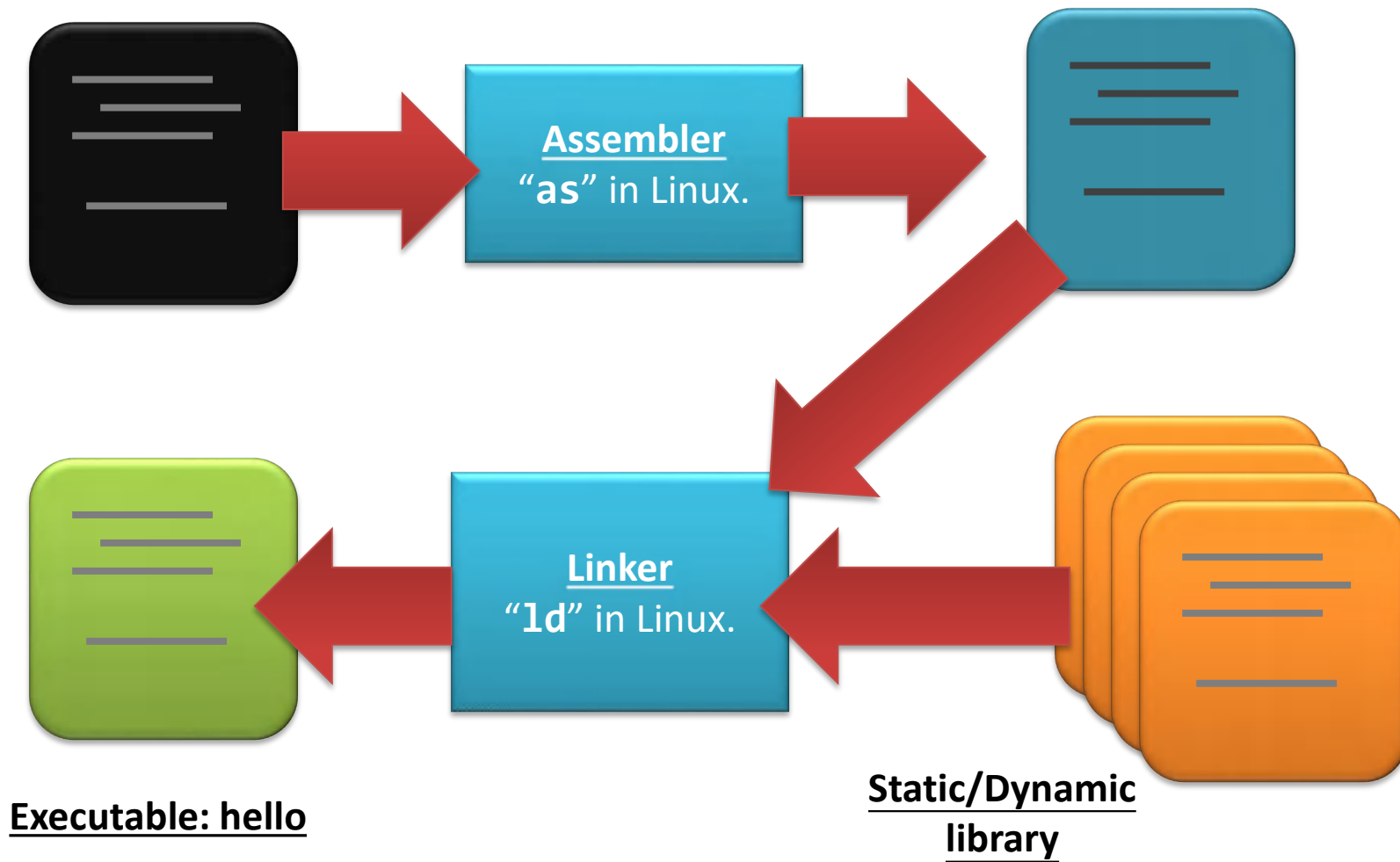




创建程序的流程

Assembly code: hello.s

Object code: hello.o





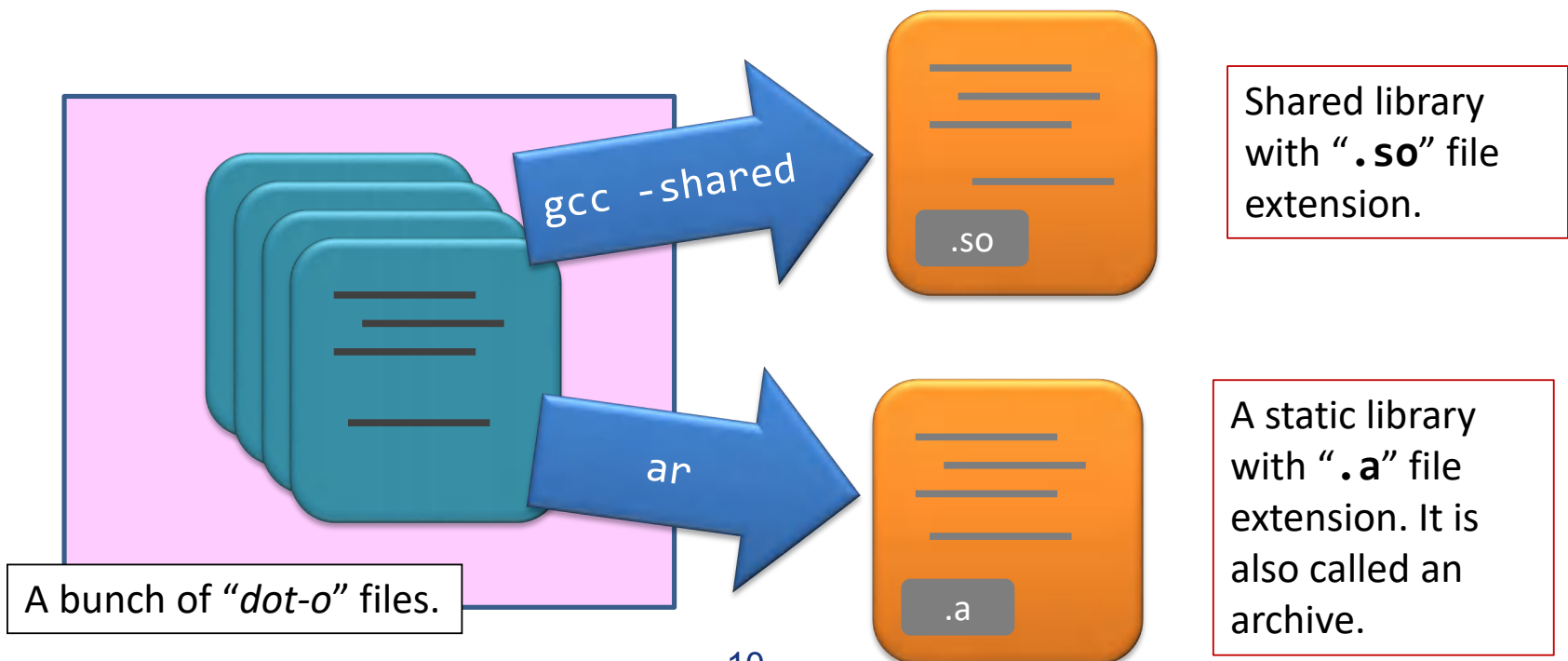
装配和链接

- The assembler assembles “**hello.s**” and generates an object code “**hello.o**”
 - A step closer to machine code
 - Try: “**as hello.s -o hello.o**”
- The linker puts together all object files as well as the libraries
 - There are two kinds of libraries: **statically-linked** and **dynamically-linked** ones



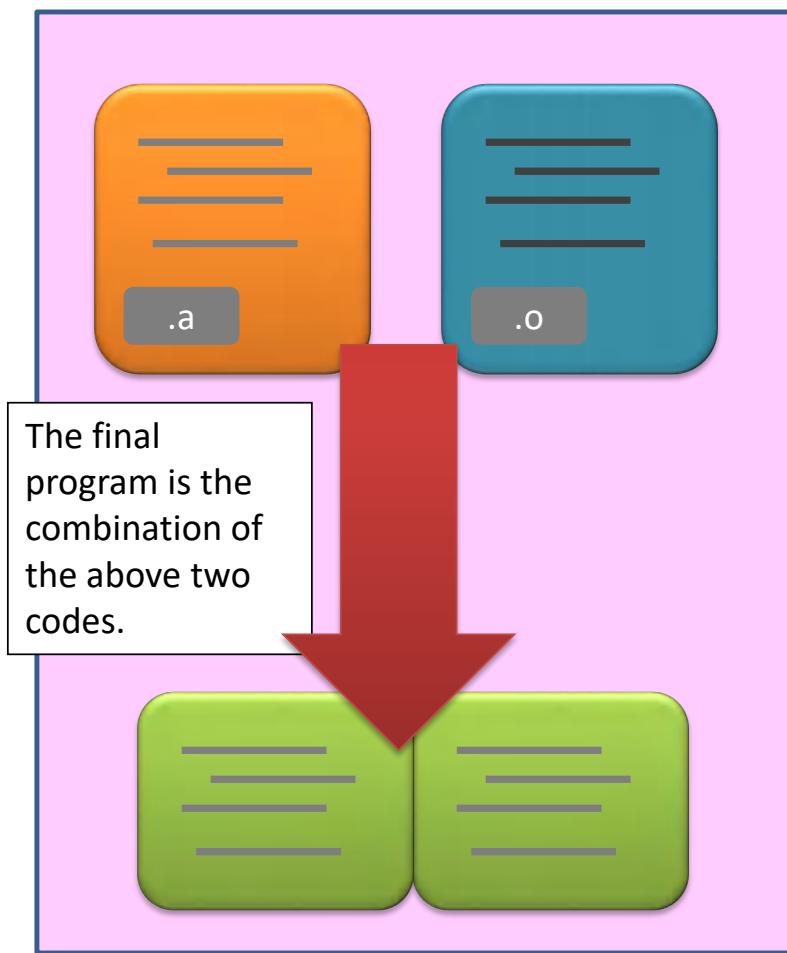
库文件

- A library file is...
 - just a bunch of function implementations.
 - for the linker to look for the function(s) that the target C program needs.

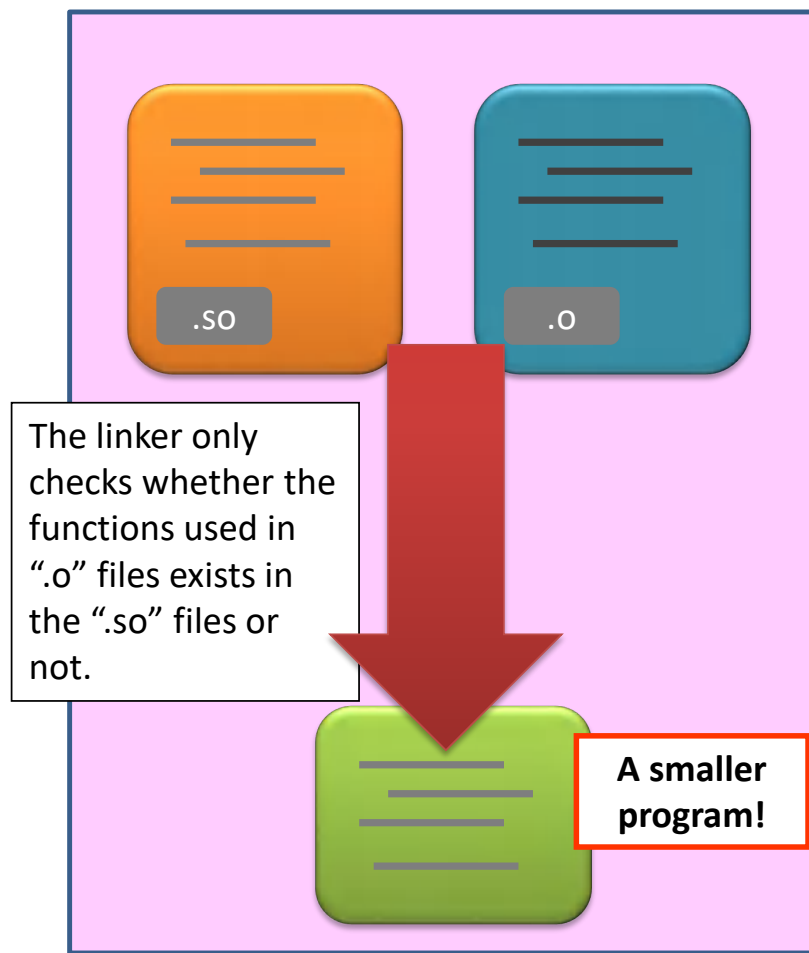




库文件



Linking with static library file.



Linking with dynamic library file.



如何编译多个源文件

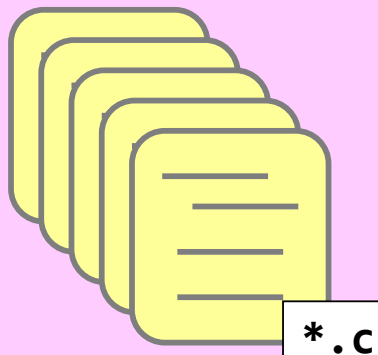
- **gcc** by default hides all the intermediate steps.
 - Executable: “**gcc -o hello hello.c**” generates “**hello**” directly.
 - Object code: “**gcc -c hello.c**” generates “**hello.o**” directly.
- How about working with multiple files?



如何编译多个源文件

Remember, below shows one of the solution.

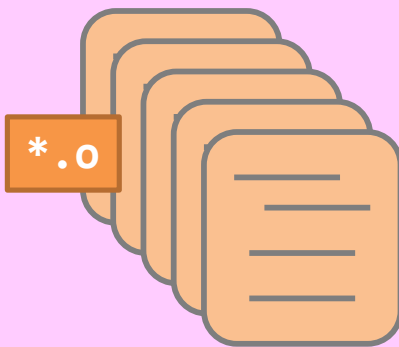
Step 1.



Prepare all the source files.
Important: there must be one and only one file containing the main function.

Step 2.

```
$ gcc -c code.c  
.....
```



Compile them into object codes one by one.

Step 3.

```
$ gcc -o prog *.o
```



Construct the program together with all the object codes.



程序是什么

- A program is just an executable file!
 - It is static;
 - It may be associated with dynamically-linked files;
 - “*.so” in Linux and “*.dll” in Windows.
- It may be compiled from more than one file



进程的概念

- ❖ 操作系统执行作为进程运行的各种程序。
- ❖ 进程 - 正在执行的程序；进程执行必须按顺序进行。
- ❖ 多部分构成
 - 程序代码，也称为文本段
 - 当前活动，包括程序计数器、处理器寄存器
 - 包含临时数据的堆栈
 - 函数参数、返回地址、局部变量
 - 包含全局变量的数据部分
 - 包含在运行时动态分配的内存的堆

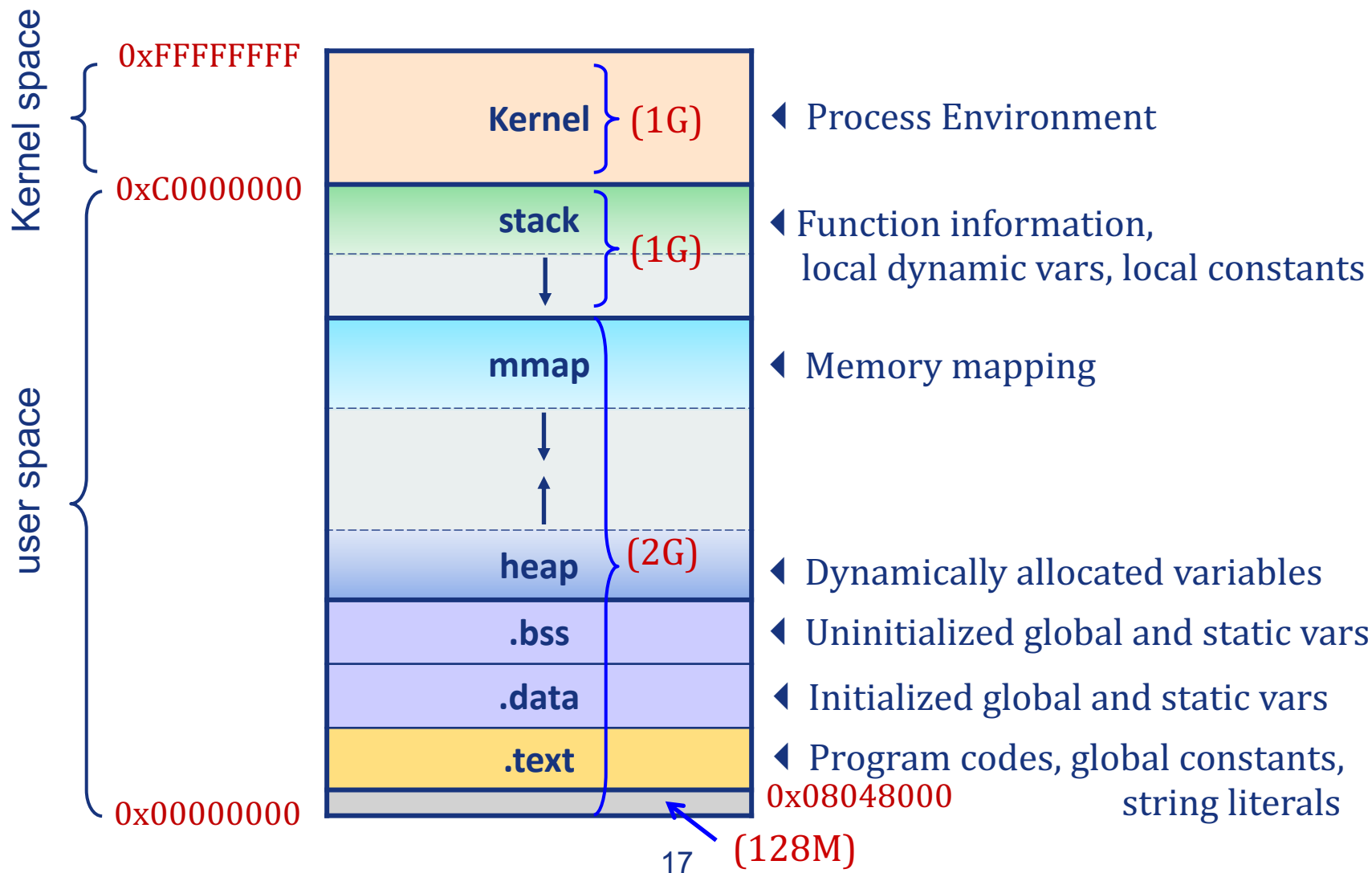


进程的概念

- ❖ 程序是存储在磁盘上的被动实体（可执行文件）；进程处于活动状态
 - 当一个可执行文件加载到内存中时，程序变成进程
- ❖ 通过GUI鼠标单击、命令行输入其名称等启动程序的执行。
- ❖ 一个程序可以是多个进程
 - 考虑多个用户执行同一个程序

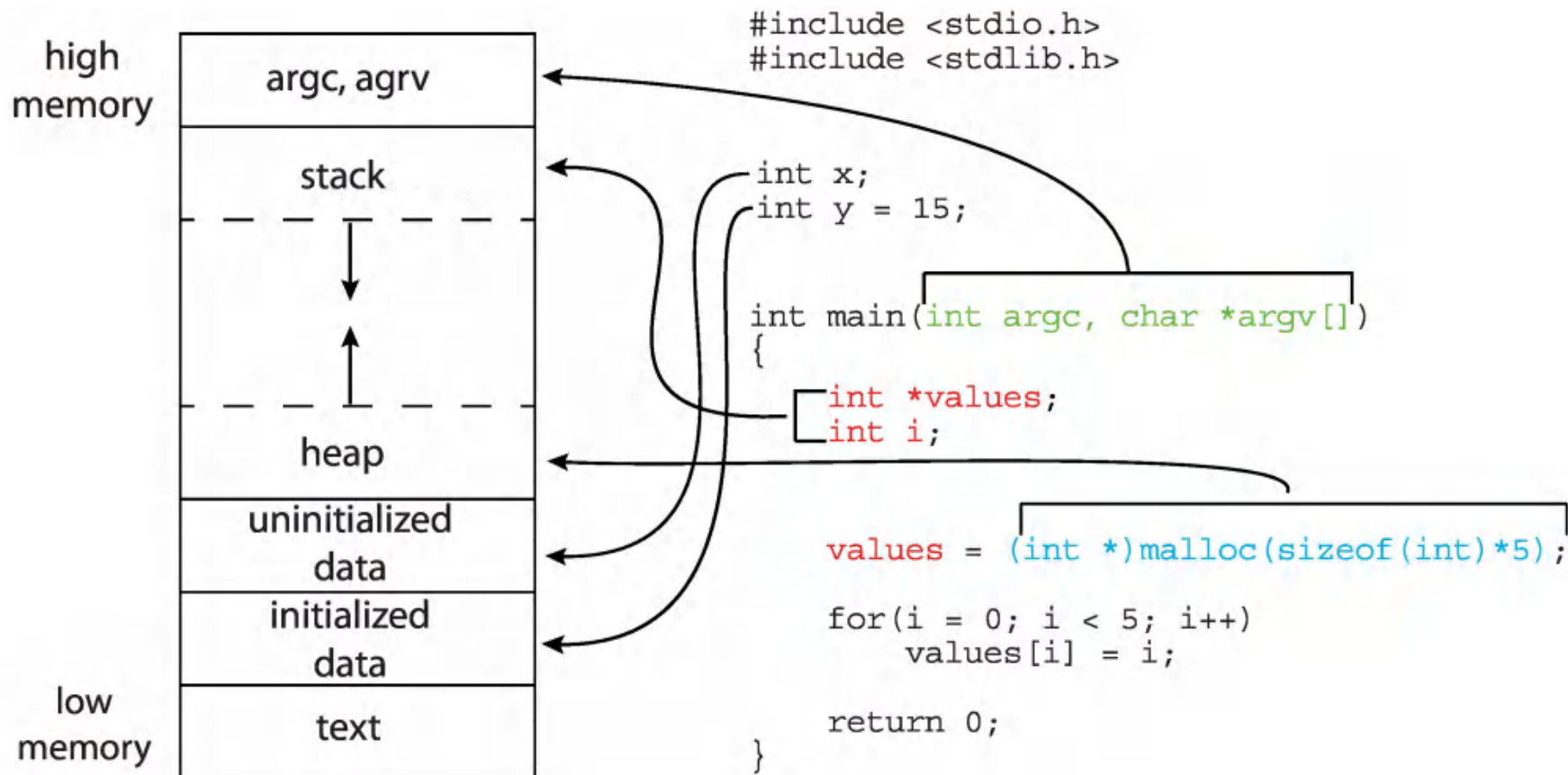


进程在内存中的布局





进程的概念



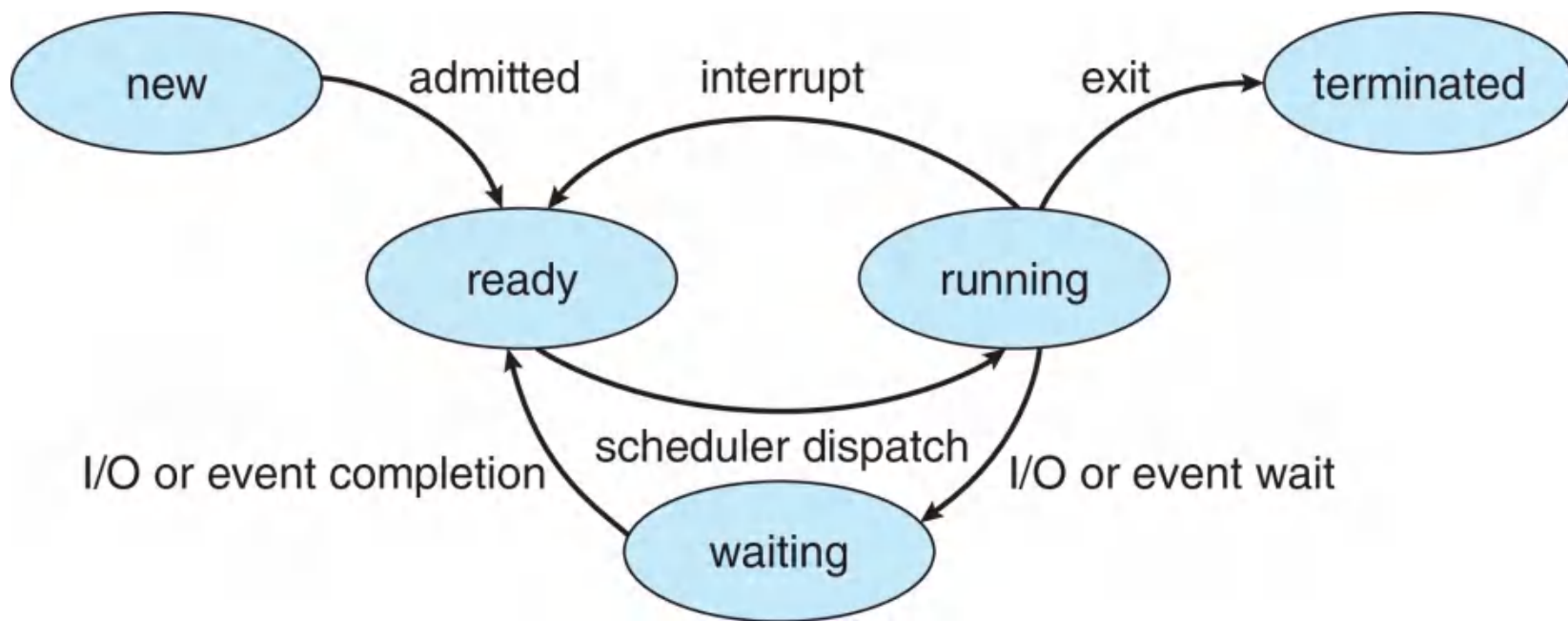


进程的状态

- ❖ 当进程执行时，它会改变状态
 - 新建：正在创建进程
 - 正在运行：正在执行指令
 - 等待：进程正在等待某个事件发生
 - 就绪：进程正在等待分配给处理器
 - 已终止：进程已完成执行



进程的状态



5状态进程模型



进程状态

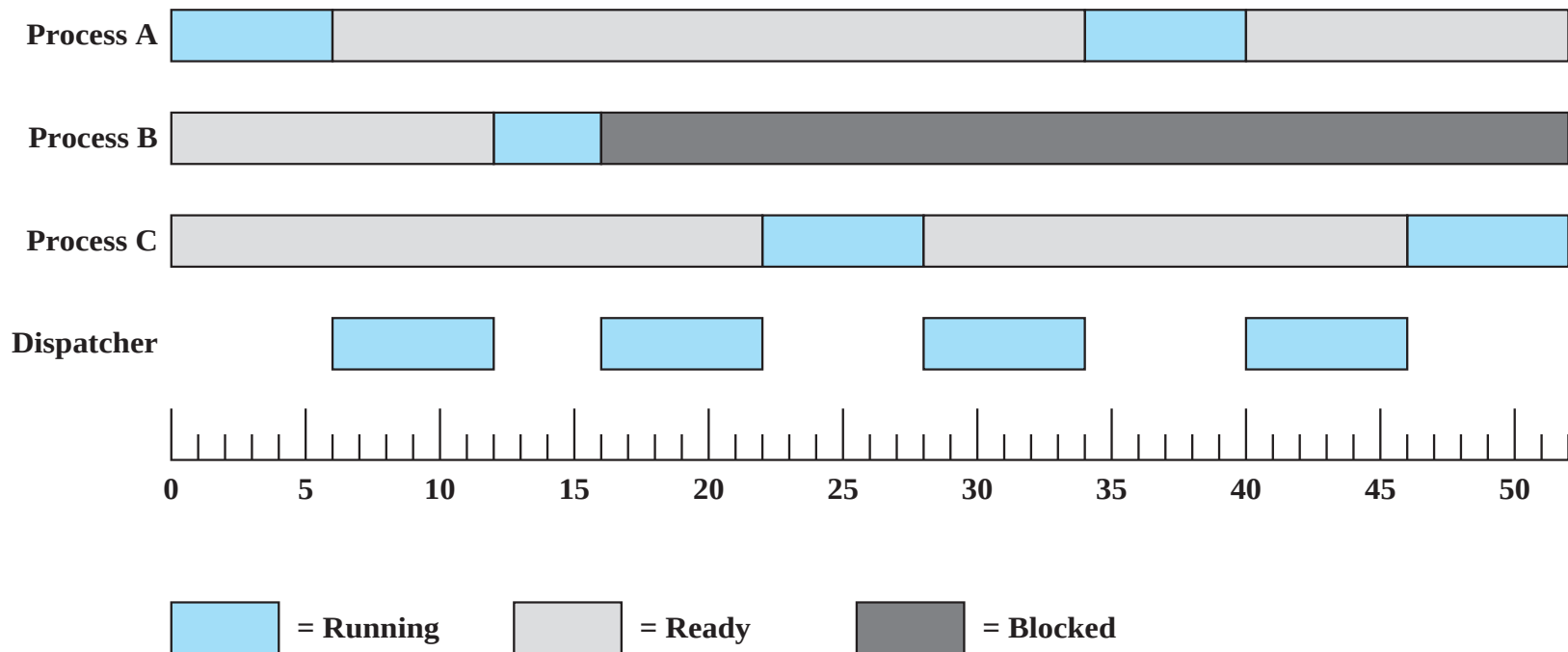


Figure 3.7 Process States for Trace of Figure 3.4



进程销毁的原因

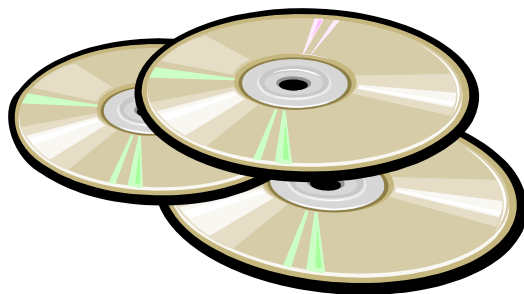
- **正常完成**：进程自行执行一个操作系统服务调用，表示它已经结束运行。
- **超过时限**：进程运行时间超过规定的时限，可测量多种时间，如总运行时间、执行时间等。
- **无可可用内存**：系统无法满足进程需要的内存空间。
- **超出范围**：进程试图访问不允许访问的内存单元。
- **保护错误**：进程试图使用不允许的资源或文件，或使用方式不正确。
- **算术错误**：进程试图进行被禁止的计算，如除以零等。
- **时间超出**：进程等待某一事件发生的时间超过规定最大值。
- **I/O 失败**：输入或输出期间发生错误，如找不到文件等。
- **无效指令**：进程试图执行不存在的指令。
- **特权指令**：进程试图使用为操作系统保留的指令。
- **数据误用**：错误类型或未初始化的数据。
- **操作员或操作系统干涉**：操作员或操作系统因某些原因终止进程。
- **父进程终止**：父进程终止时，系统可能自动终止其子进程。
- **父进程请求**：父进程有权终止其子进程



挂起的进程

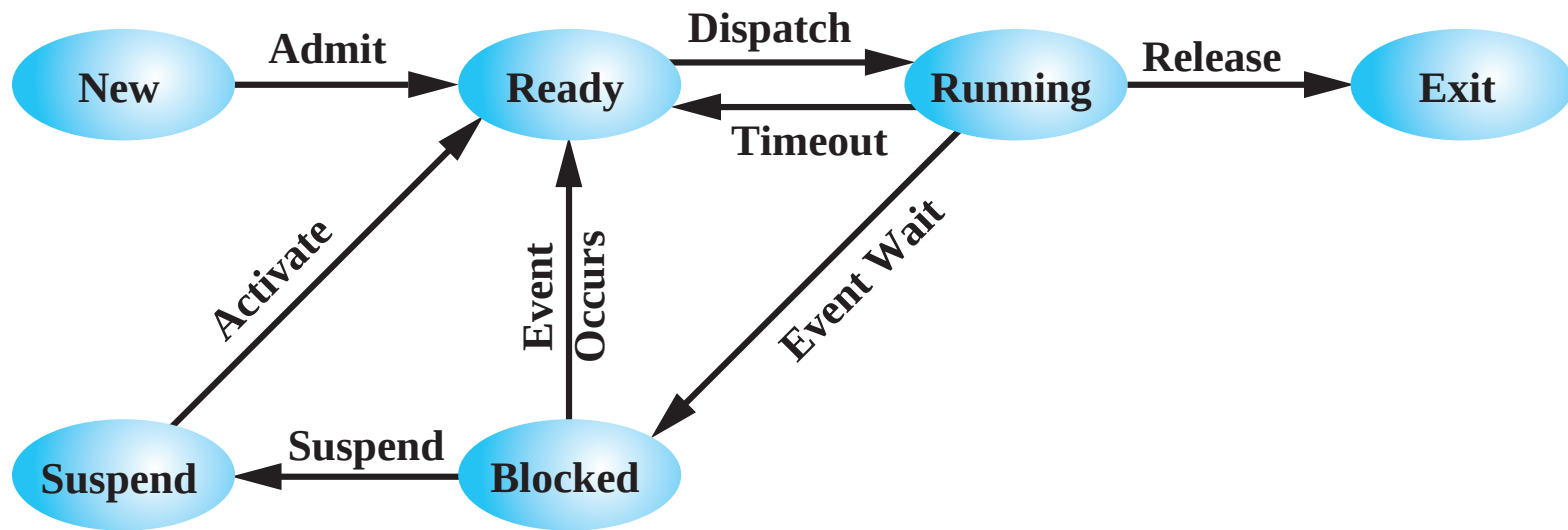
■ 交换需求

- 将内存中的一个进程的一部分或者全部移到磁盘上；
- 当内存中没有进程处于就绪状态时，操作系统将阻塞的进程交换到磁盘中，进入挂起队列；





单挂起状态模型

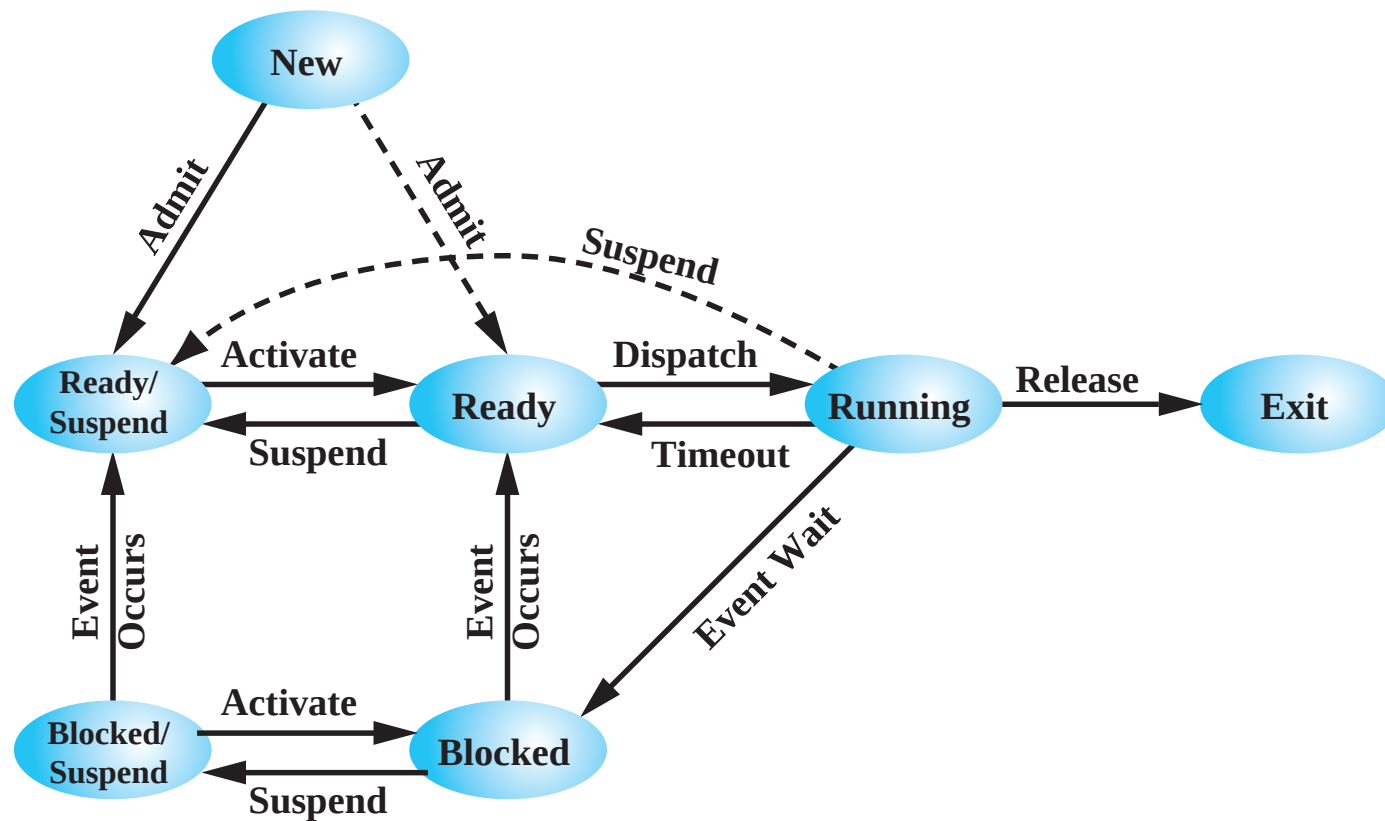


(a) With One Suspend State

Figure 3.9 Process State Transition Diagram with Suspend States



双挂起状态的进程模型



(b) With Two Suspend States

Figure 3.9 Process State Transition Diagram with Suspend States



挂起进程的特征

- ❖ 进程不能立即执行；
- ❖ 进程可能有也可能没有等待一个事件；
- ❖ 为阻止该进程执行，可通过代理使其处于挂起状态，代理可以是进程本身，也可以是父进程或者操作系统
- ❖ 除非代理程序显式地命令系统进行状态轮换，否则该进程无法从这一状态转移。



进程挂起的原因

Swapping

The OS needs to release sufficient main memory to bring in a process that is ready to execute.

Other OS reason

The OS may suspend a background or utility process or a process that is suspected of causing a problem.

Interactive user request

A user may wish to suspend execution of a program for purposes of debugging or in connection with the use of a resource.

Timing

A process may be executed periodically (e.g., an accounting or system monitoring process) and may be suspended while waiting for the next time interval.

Parent process request

A parent process may wish to suspend execution of a descendent to examine or modify the suspended process, or to coordinate the activity of various descendants.



进程和描述

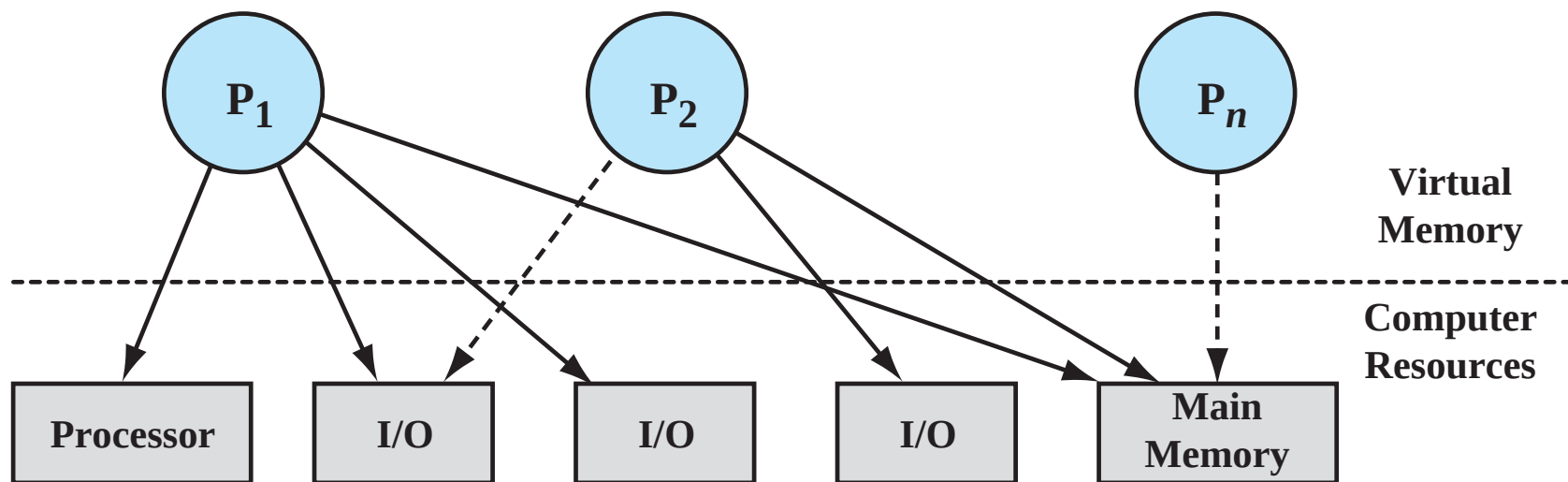


Figure 3.10 Processes and Resources (resource allocation at one snapshot in time)



进程和描述

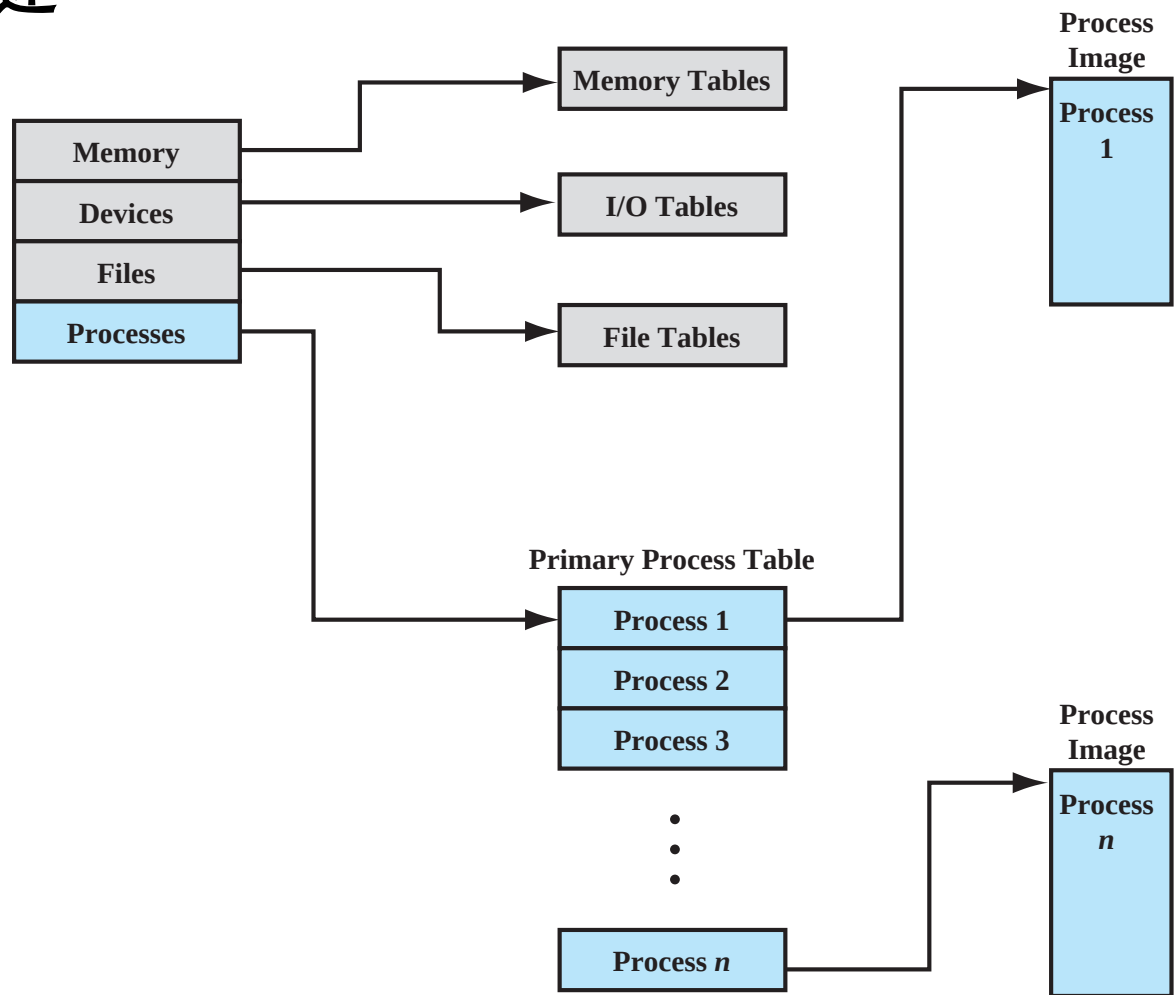


Figure 3.11 General Structure of Operating System Control Tables



内存表

- ❖ Used to keep track of both main (real) and secondary (virtual) memory
- ❖ Processes are maintained on secondary memory using some sort of virtual memory or simple swapping mechanism

Must include:

allocation of main memory to processes

allocation of secondary memory to processes

protection attributes of blocks of main or virtual memory

information needed to manage virtual memory



I/O表

- ❖ **Used by the OS to manage the I/O devices and channels of the computer system**
- ❖ **At any given time, an I/O device may be available or assigned to a particular process**

If an I/O operation is in progress, the OS needs to know:

- the status of the I/O operation
- the location in main memory being used as the source or destination of the I/O transfer



文件表

- ❖ **Information may be maintained and used by a file management system**
 - in which case the OS has little or no knowledge of files
- In other operating systems, much of the detail of file management is managed by the OS itself

These tables provide information about:

- existence of files
- location on secondary memory
- current status
- other attributes



进程表

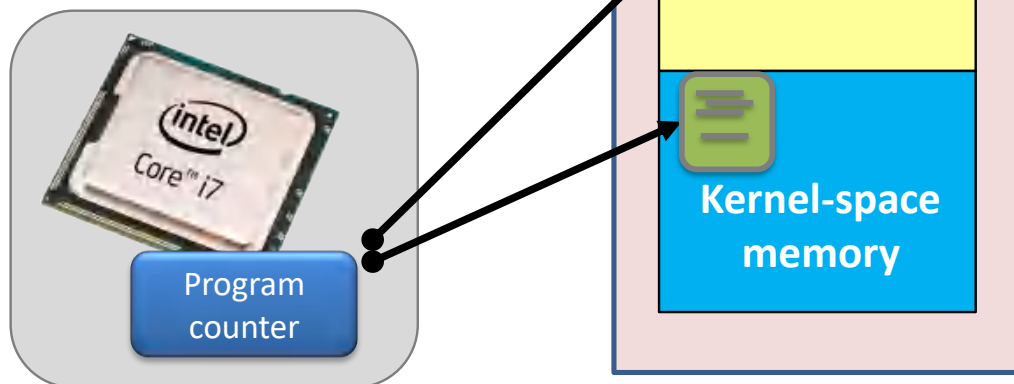
- ❖ 必须进行维护以管理进程
- ❖ 必须直接或间接引用内存、I/O和文件
- ❖ 操作系统必须能够访问表本身，因此需要进行内存管理

操作系统在什么地方创建进程表？



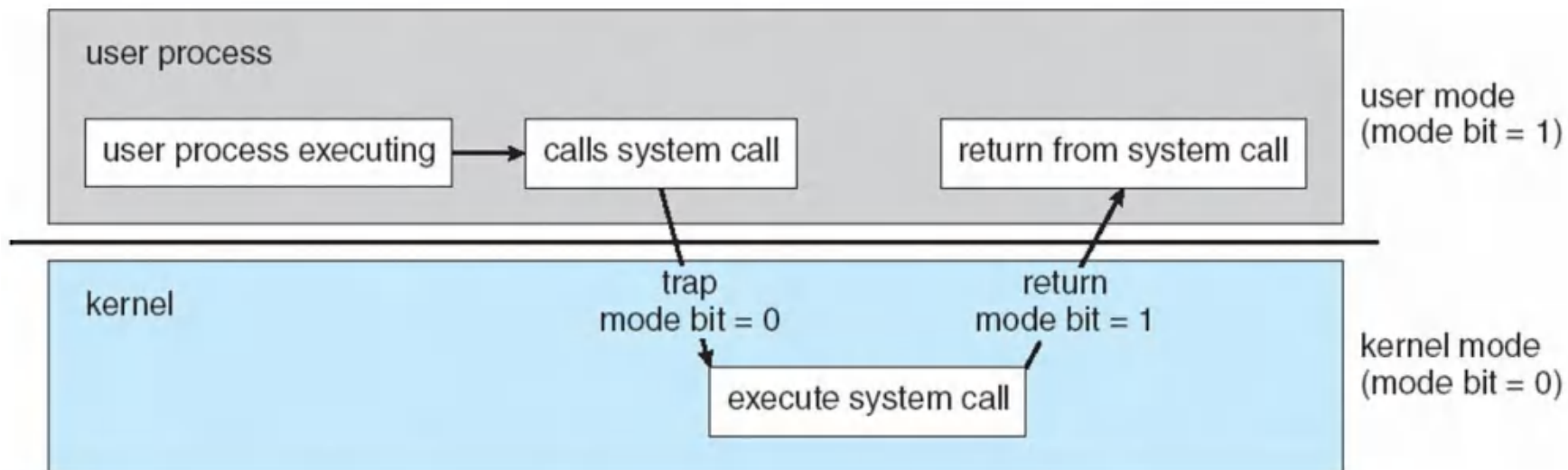
Getpid()

- What happens...
 - When the process is calling the system call “**getpid()**”
- Where to get the PID
 - PCB (in kernel-space memory)
- The CPU switches from the user-space to the kernel-space, and reads the PID





Getpid()

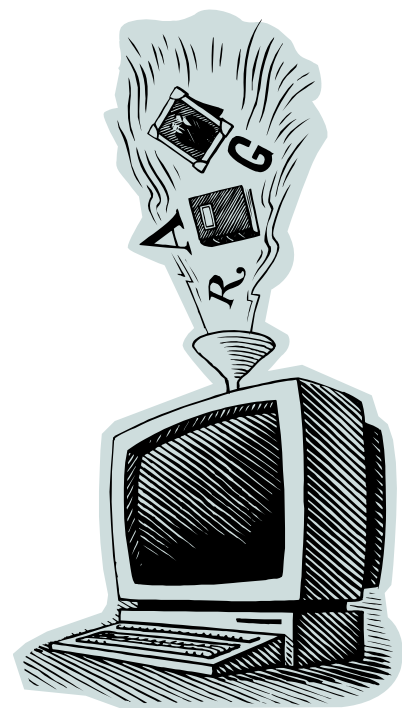


Another question: How much time was spent in each part?



进程控制块 (PCB)

- ❖ 是操作系统控制和协调各种活动进程所需要的信息;
- ❖ 在进程控制块中包含了进程控制信息;





进程控制块 (PCB)

与每个进程相关的信息或者进程的表示
(也称为任务控制块)

- ❖ 进程状态 - 正在运行、等待等。
- ❖ 程序计数器 - 下一个执行指令的位置
- ❖ CPU寄存器 - 所有以进程为中心的寄存器的内容
- ❖ CPU调度信息-优先级、调度队列指针
- ❖ 内存管理信息 - 分配给进程的内存
- ❖ 记账信息 - 使用的CPU、启动后经过的时钟时间、时间限制
- ❖ I/O状态信息 - 分配给进程的I/O设备、打开的文件列表

process state
process number
program counter
registers
memory limits
list of open files
...



进程控制块的作用

- ❖ 进程控制块是操作系统最重要的数据结构;
- ❖ 每个进程控制块都包含了操作系统所需进程的所有信息;
 - 操作系统中的每一个模块, 包括: 资源调度、资源分配、中断处理、性能监控和分析模块都可以读取和修改它们;
 - 定义了操作系统的状态;
- ❖ 困难的不是访问, 而是保护:
 - 一个进程中的错误可能会破坏进程控制块, 进而破坏系统对受影响的进程的管理能力。
 - 进程控制块结构或语义中的设计变化可能会影响到操作系统中的很多模块;



进程控制块 (PCB)

❖ Linux中的进程控制块位于source code/include/linux

```
640 struct task_struct {
641 #ifdef CONFIG_THREAD_INFO_IN_TASK
642     /*
643      * For reasons of header soup (see current_thread_info()), this
644      * must be the first element of task_struct.
645      */
646     struct thread_info      thread_info;
647 #endif
648     /* -1 unrunnable, 0 runnable, >0 stopped: */
649     volatile long           state;
650
651     /*
652      * This begins the randomizable portion of task_struct. Only
653      * scheduling-critical items should be added above here.
654      */
655     randomized_struct_fields_start
656
657     void                    *stack;
658     refcount_t              usage;
659     /* Per task flags (PF_*), defined further below: */
660     unsigned int            flags;
661     unsigned int            ptrace;
662
663 #ifdef CONFIG_SMP
664     int                     on_cpu;
665     struct __call_single_node wake_entry;
666 #ifdef CONFIG_THREAD_INFO_IN_TASK
667     /* Current CPU: */
668     unsigned int            cpu;
669 #endif
670     unsigned int            wakee_flips;
671     unsigned long           wakee_flip_decay_ts;
672     struct task_struct      *last_wakee;
673
674     /*
```

Demo



线程

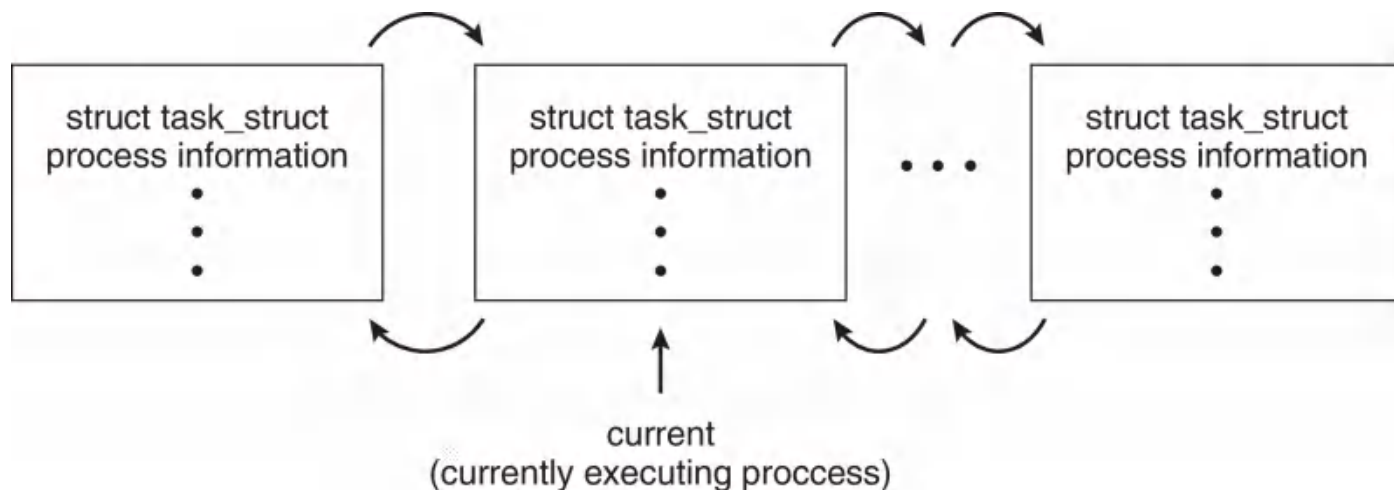
- ❖ 到目前为止，进程只有一个执行线程
- ❖ 考虑每个进程具有多个程序计数器
 - 可以同时执行多个线程（多条指令流）
 - 多线程控制
- ❖ 然后必须在PCB中存储线程详细信息和多个程序计数器
- ❖ 在第4章中详细探讨



Linux 进程控制块 (PCB)

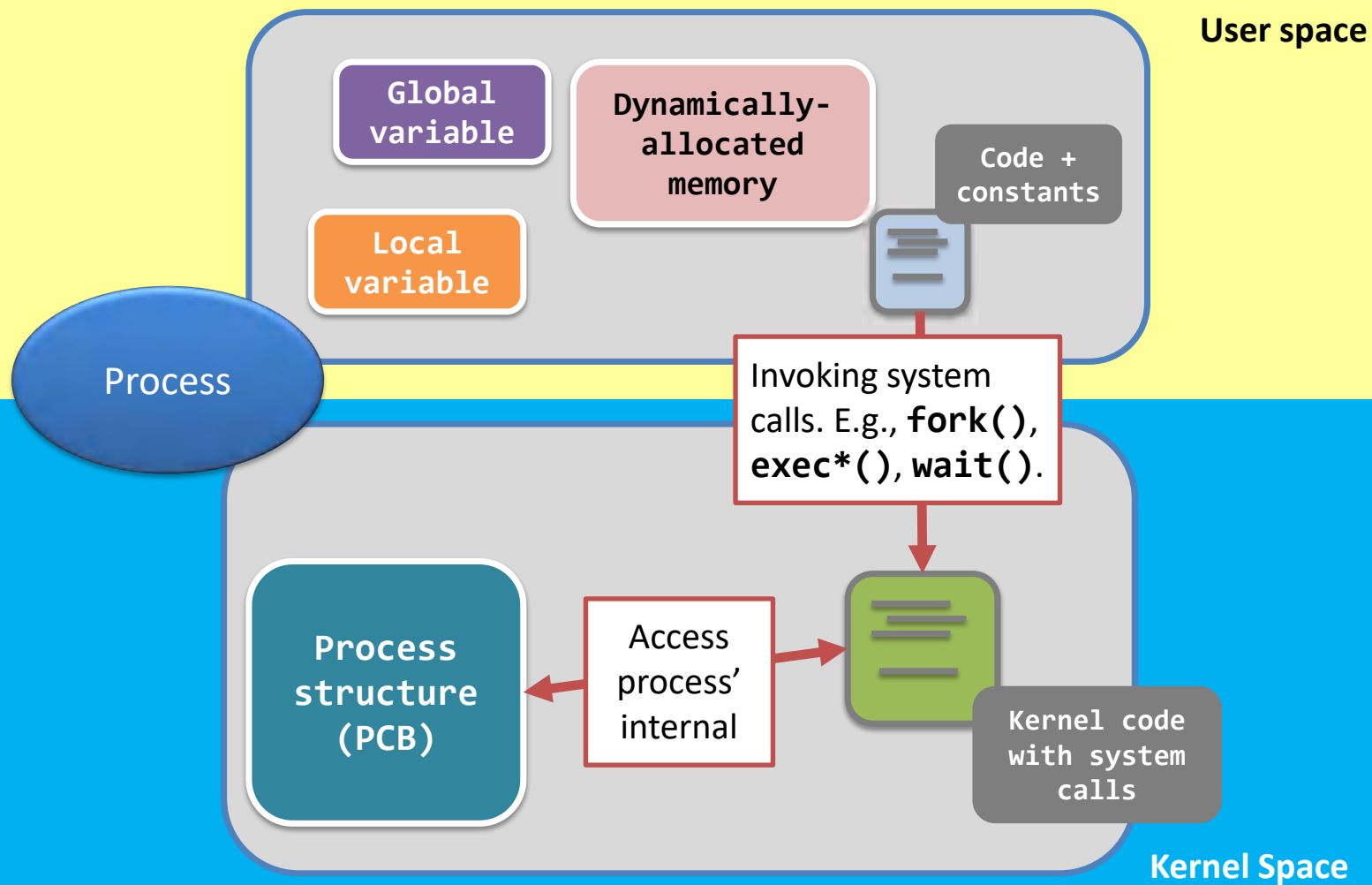
由C结构task_struct表示

```
{ pid t_pid; /*进程标识符*/  
  unsigned int time_slice /*调度信息*/  
  struct task_struct* 父进程; /*此进程的父级*/  
  struct list_head children; /*此进程的子进程*/  
  struct *files; /*打开文件列表*/  
  struct mm_struct*mm; /*此进程的地址空间*/  
  ... }
```





Relationship between Process Data & PCB





进程映像元素

User Data

The modifiable part of the user space. May include program data, a user stack area, and programs that may be modified.

User Program

The program to be executed.

Stack

Each process has one or more last-in-first-out (LIFO) stacks associated with it. A stack is used to store parameters and calling addresses for procedure and system calls.

Process Control Block

Data needed by the OS to control the process (see Table 3.5).



进程典型元素

Process Identification

Identifiers

Numeric identifiers that may be stored with the process control block include

- Identifier of this process
- Identifier of the process that created this process (parent process)
- User identifier

Processor State Information

User-Visible Registers

A user-visible register is one that may be referenced by means of the machine language that the processor executes while in user mode. Typically, there are from 8 to 32 of these registers, although some RISC implementations have over 100.

Control and Status Registers

These are a variety of processor registers that are employed to control the operation of the processor. These include

- **Program counter:** Contains the address of the next instruction to be fetched
- **Condition codes:** Result of the most recent arithmetic or logical operation (e.g., sign, zero, carry, equal, overflow)
- **Status information:** Includes interrupt enabled/disabled flags, execution mode

Stack Pointers

Each process has one or more last-in-first-out (LIFO) system stacks associated with it. A stack is used to store parameters and calling addresses for procedure and system calls. The stack pointer points to the top of the stack.



进程控制元素

Process Control Information

Scheduling and State Information

This is information that is needed by the operating system to perform its scheduling function. Typical items of information:

- Process state:** Defines the readiness of the process to be scheduled for execution (e.g., running, ready, waiting, halted).
- Priority:** One or more fields may be used to describe the scheduling priority of the process. In some systems, several values are required (e.g., default, current, highest-allowable)
- Scheduling-related information:** This will depend on the scheduling algorithm used. Examples are the amount of time that the process has been waiting and the amount of time that the process executed the last time it was running.
- Event:** Identity of event the process is awaiting before it can be resumed.

Data Structuring

A process may be linked to other process in a queue, ring, or some other structure. For example, all processes in a waiting state for a particular priority level may be linked in a queue. A process may exhibit a parent-child (creator-created) relationship with another process. The process control block may contain pointers to other processes to support these structures.

Interprocess Communication

Various flags, signals, and messages may be associated with communication between two independent processes. Some or all of this information may be maintained in the process control block.

Process Privileges

Processes are granted privileges in terms of the memory that may be accessed and the types of instructions that may be executed. In addition, privileges may apply to the use of system utilities and services.

Memory Management

This section may include pointers to segment and/or page tables that describe the virtual memory assigned to this process.

Resource Ownership and Utilization

Resources controlled by the process may be indicated, such as opened files. A history of utilization of the processor or other resources may also be included; this information may be needed by the scheduler.



进程标识符

- ❖ **Each process is assigned a unique numeric identifier**
 - otherwise there must be a mapping that allows the OS to locate the appropriate tables based on the process identifier
- ❖ **Many of the tables controlled by the OS may use process identifiers to cross-reference process tables**
- ❖ **Memory tables may be organized to provide a map of main memory with an indication of which process is assigned to each region**
 - similar references will appear in I/O and file tables
- ❖ **When processes communicate with one another, the process identifier informs the OS of the destination of a particular communication**
- ❖ **When processes are allowed to create other processes, identifiers indicate the parent and descendents of each process**



处理器状态信息

Consists
of the
contents
of
processor
registers

- user-visible registers
- control and status registers
- stack pointers

**Program
status
word
(PSW)**

- contains condition codes plus other status information
- EFLAGS register is an example of a PSW used by any OS running on an x86 processor



进程控制——执行模式

❖ 用户模式

非特权模式；

用户程序运行在该模式下：

❖ 系统模式

特权模式；

也被称为控制模式和内核模式；

内核模式指的是操作系统内核，是操作系统中重要的系统功能；

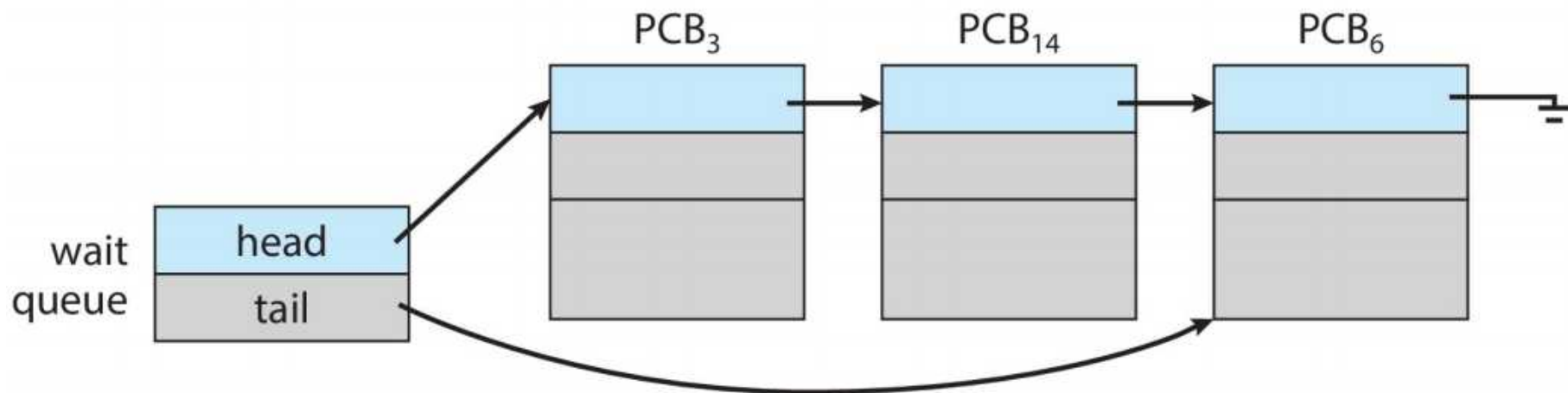
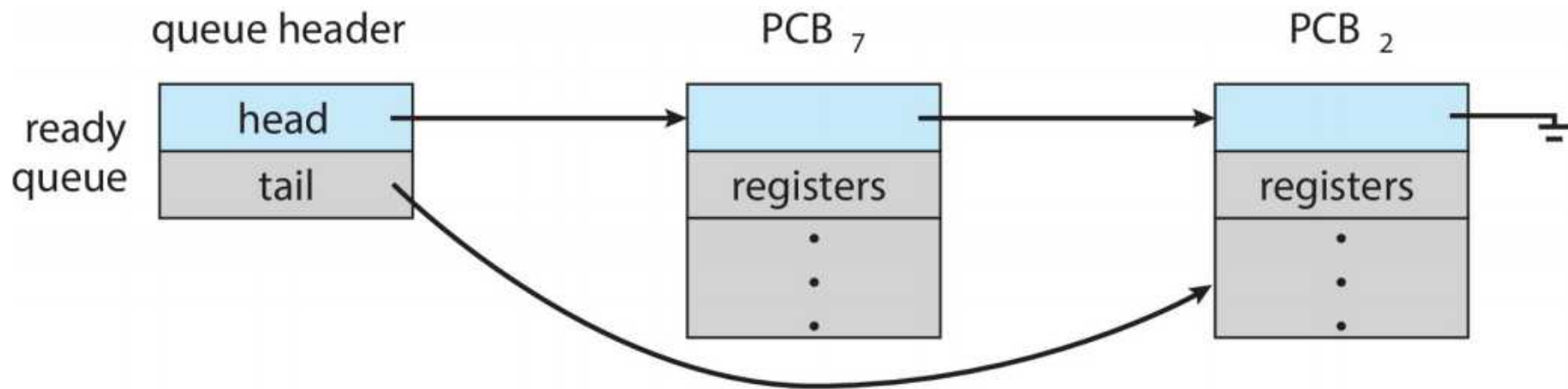


进程调度

- ❖ Process scheduler从可用进程中选择下一个在CPU核心上执行的进程
- ❖ 目标——最大限度地利用CPU，快速将进程切换到CPU核心
- ❖ 维护进程的调度队列
 - 就绪队列 - 驻留在主内存中、就绪并等待执行的所有进程的集合
 - 等待队列 - 等待事件（即I/O）的一组进程
 - 进程在各种队列之间迁移



进程调度





进程链表结构

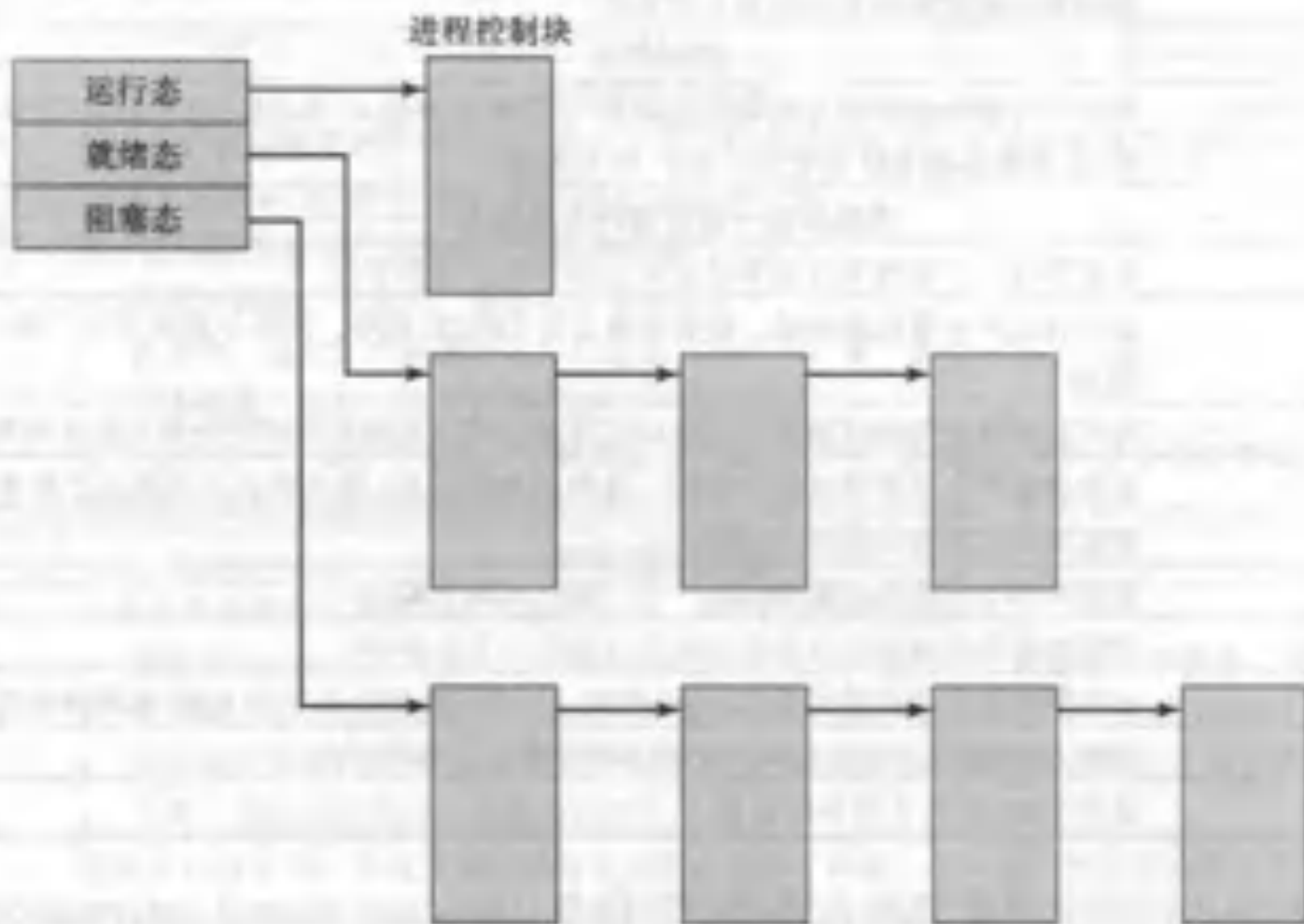
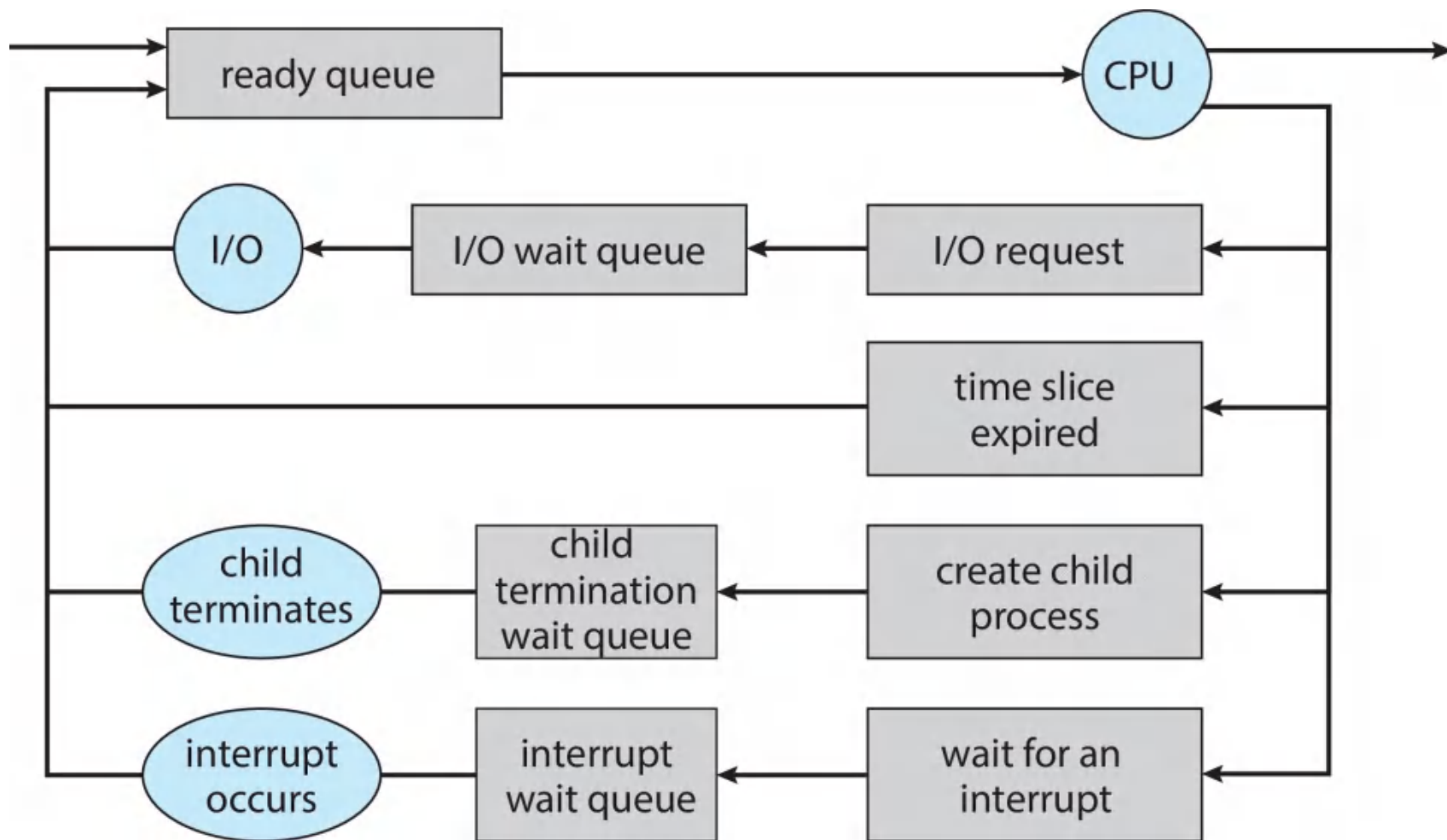


图 3.14 进程链表结构

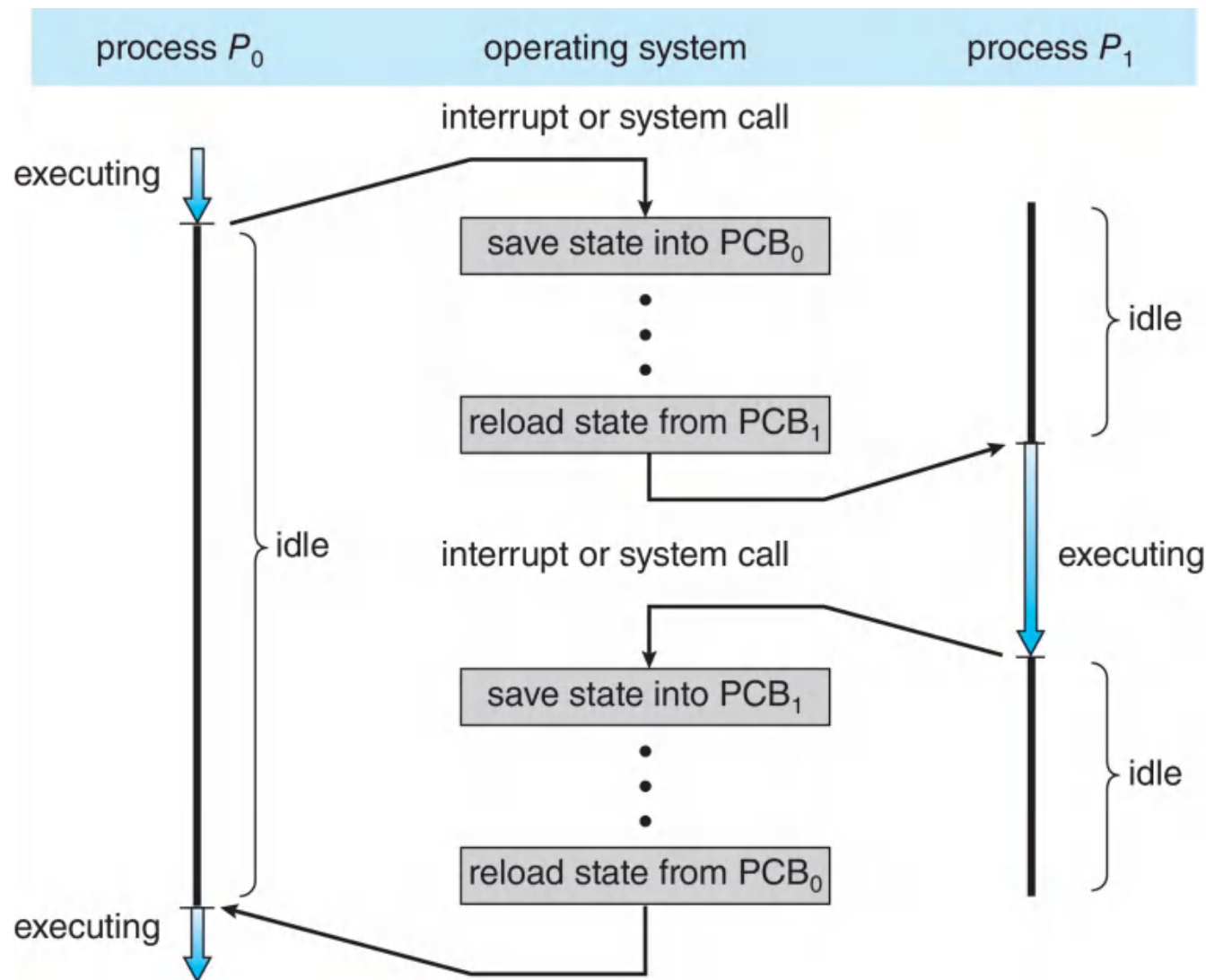


进程调度队列





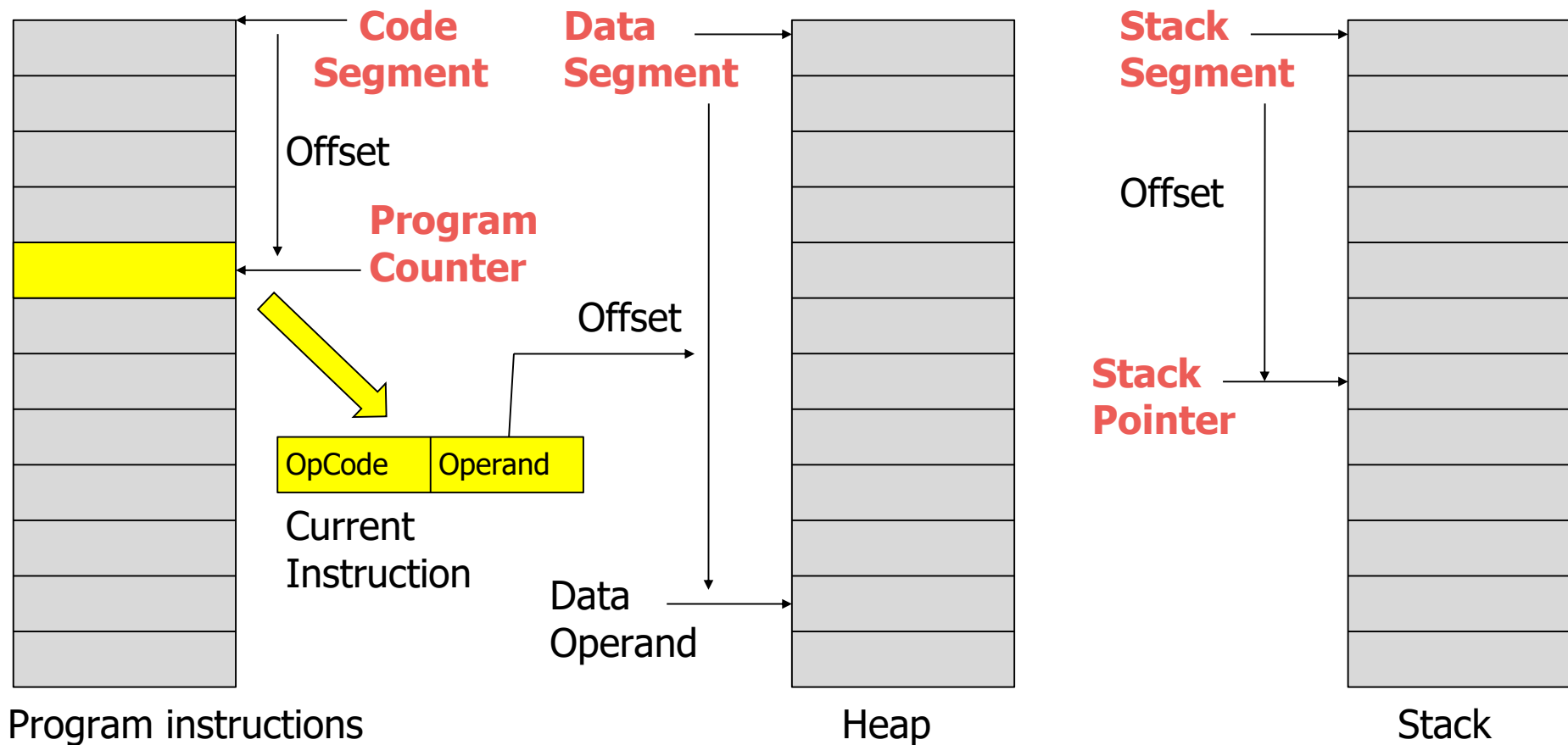
进程切换





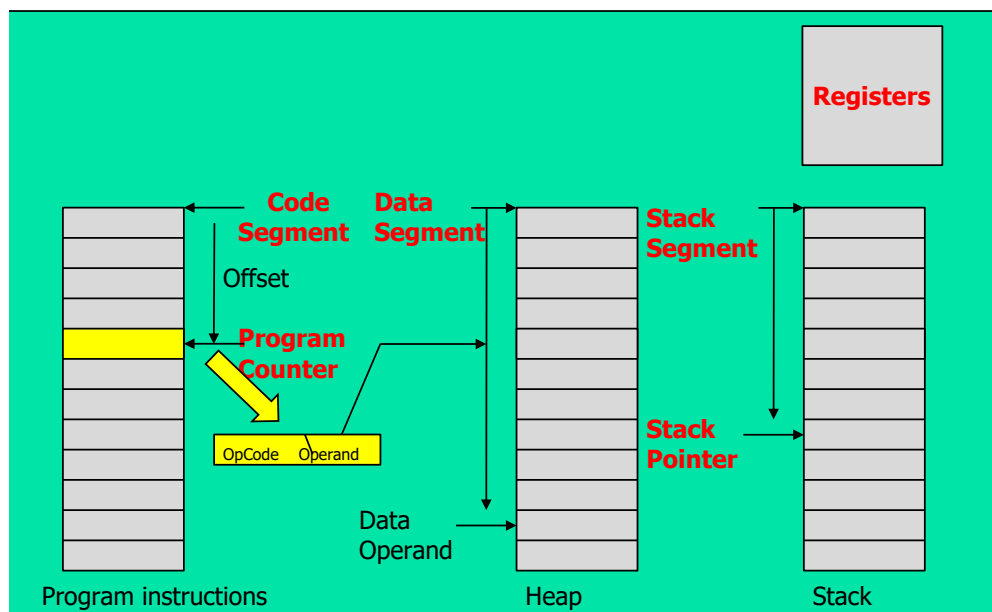
What's the **STATE** of a real CPU?

Registers

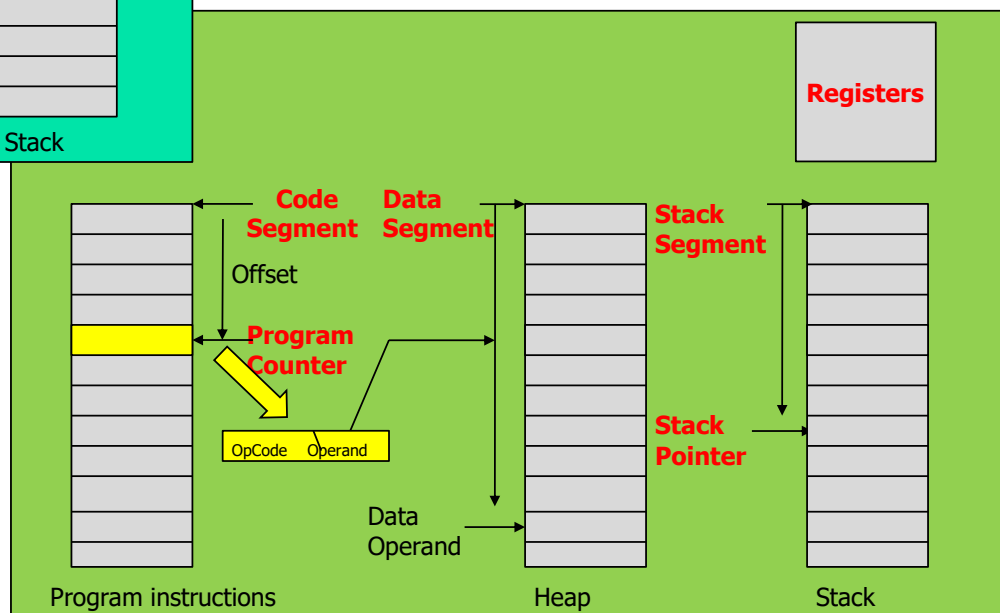




进程上下文切换



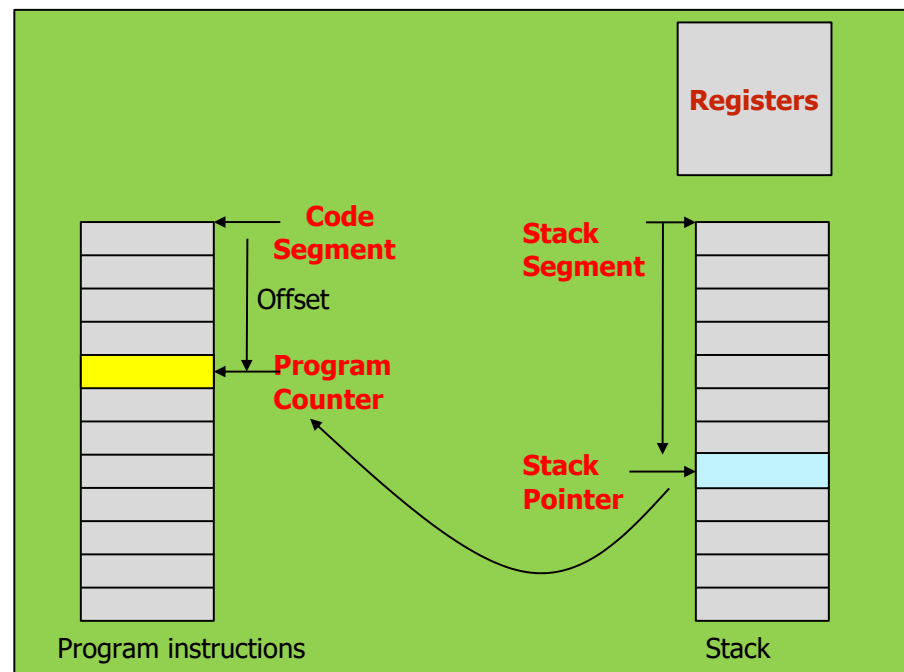
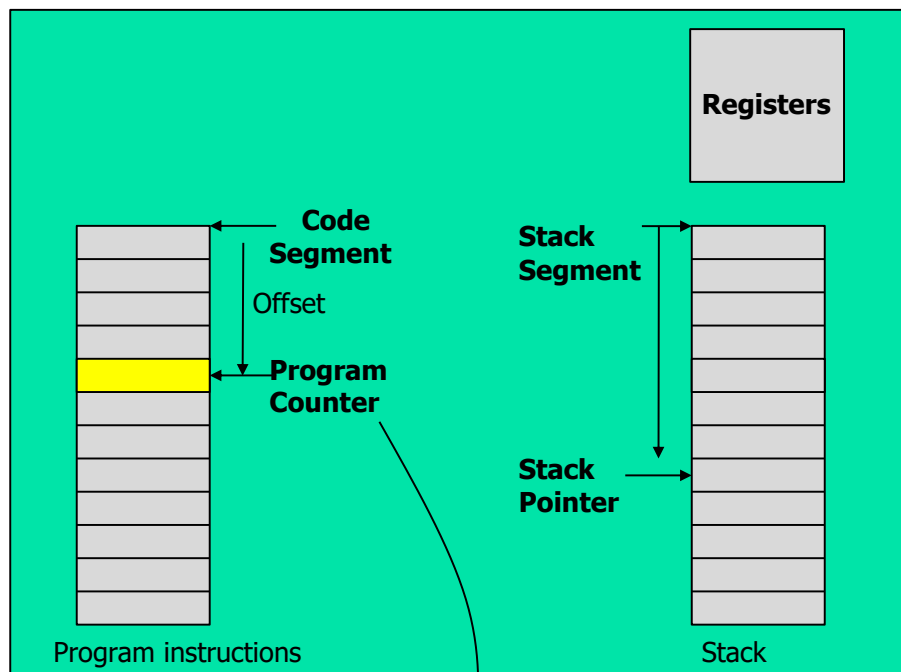
**Save State
(Context)**



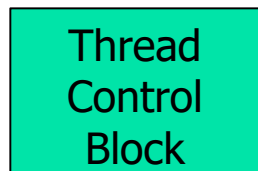
**Load State
(Context)**



进程上下文切换 (中断)

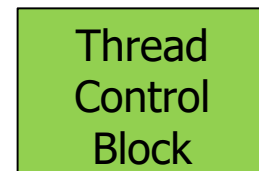


Save PC on thread stack
Jump to Interrupt handler



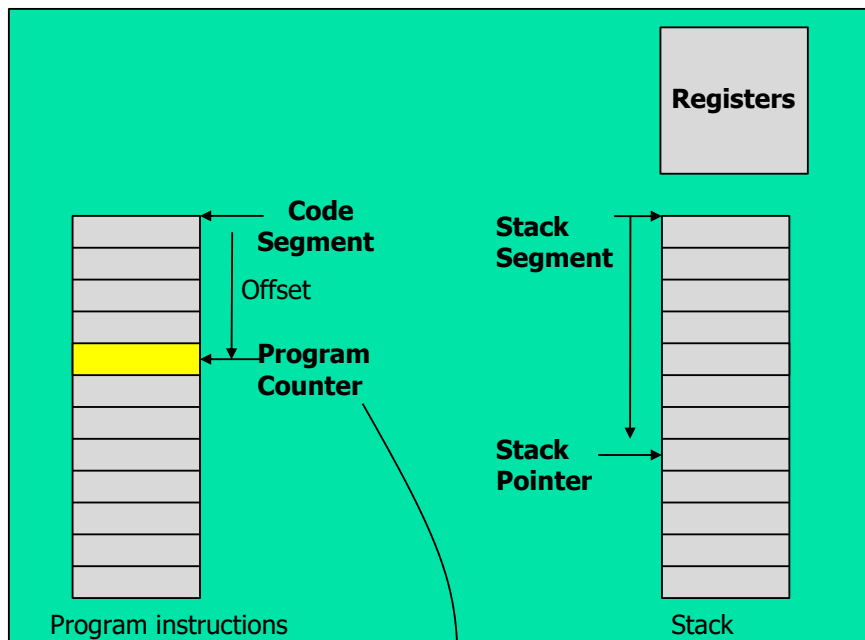
Handler

- Save thread state in thread control block (SP, registers, segment pointers, ...)
- **Handle Interrupt**
- Choose next thread
- Load thread state from control block
- Pop PC from thread stack (return from handler)
- Resume prior task





进程上下文切换 (Yield)



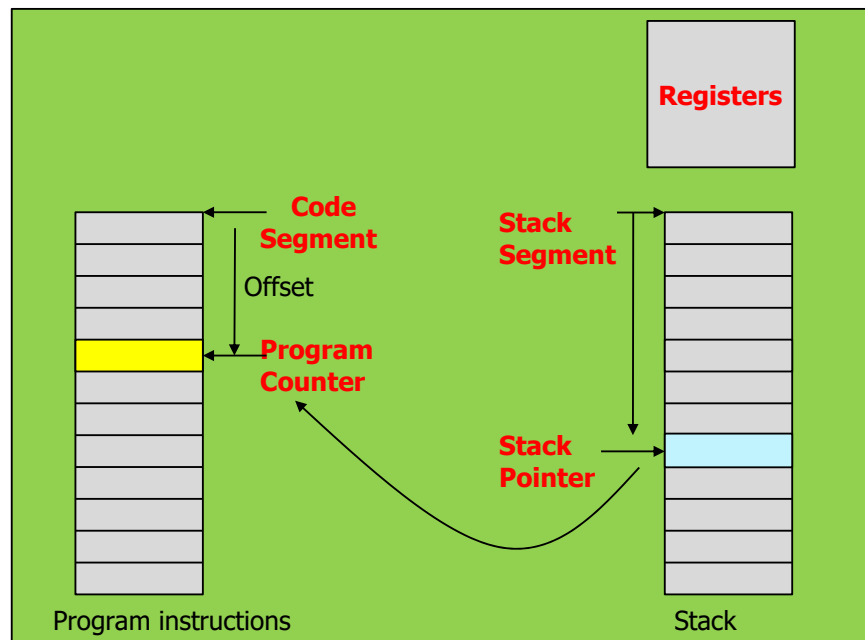
Save PC on thread stack
Jump to yield() function

Thread
Control
Block

yield()

- Save thread state in thread control block (SP, registers, segment pointers, ...)
- Choose next thread
- Load thread state from control block
- Pop PC from thread stack (return from handler)

Thread
Control
Block





系统中断

进程上下文切换可以用中断实现

❖ 中断

- 与当前进程无关的某种外部事件相关；
 - 时钟中断；
 - I/O 中断
 - 内存失效
- 时间片
 - 即进程中断前可以执行的最大时间段；

❖ 陷阱

- 运行中进程产生的错误或者异常；
- 操作系统确定错误或异常的志明程度；
 - 致命时，当前正运行进程设置为退出状态，请切换进程；
 - 非致命时，操作系统的动作将取决于错误的性质和操作系统设计；



进程状态的变化

- The steps in a full process switch are:

save the context of the processor



update the process control block of the process currently in the Running state



move the process control block of this process to the appropriate queue



select another process for execution



update the process control block of the process selected



update memory management data structures



restore the context of the processor to that which existed at the time the selected process was last switched out

If the currently running process is to be moved to another state (Ready, Blocked, etc.), then the OS must make substantial changes in its environment



移动系统中的多任务处理

- ❖ 某些移动系统（如早期版本的iOS）只允许运行一个进程，其他程序暂停运行
- ❖ 由于屏幕不稳定，iOS提供有限形式的多任务
 - 单个前台进程-通过用户界面控制
 - 多个后台进程 - 内存中，正在运行，但不在显示屏上，并且有限制
 - 限制包括单个、短任务、接收事件通知、特定的长时间运行任务（如音频播放）
- ❖ Android运行后台限制较少
 - 后台进程使用服务执行任务
 - 即使后台进程挂起，服务也可以继续运行
 - 服务没有用户界面，内存使用量小



进程的操作



进程操作

- Process
 - It associates with all the files opened by that process.
 - It attaches to all the memory that is allocated for it.
 - It contains every accounting information,
 - running time, current memory usage, who owns the process, etc.
- You couldn't operate any things without processes.

Demo



进程操作

- System must provide mechanisms for:
 - process **identification**
 - process **creation**
 - program **execution**
 - process **termination**
- Some basic and important system calls
 - **getpid()**
 - **fork()**
 - **exec*()**
 - **wait()**
 - **exit()**



进程操作

- How can we identify processes?
 - Each process is given an unique ID number, and is called the **process ID**, or the **PID**.
 - The system call, **getpid()**, prints the PID of the calling process.

```
#include <stdio.h>    // printf()
#include <unistd.h>    // getpid()

int main(void) {
    printf("My PID is %d\n", getpid() );
}
```

```
$ ./getpid
My PID is 1234
$ ./getpid
My PID is 1235
$ ./getpid
My PID is 1237
```



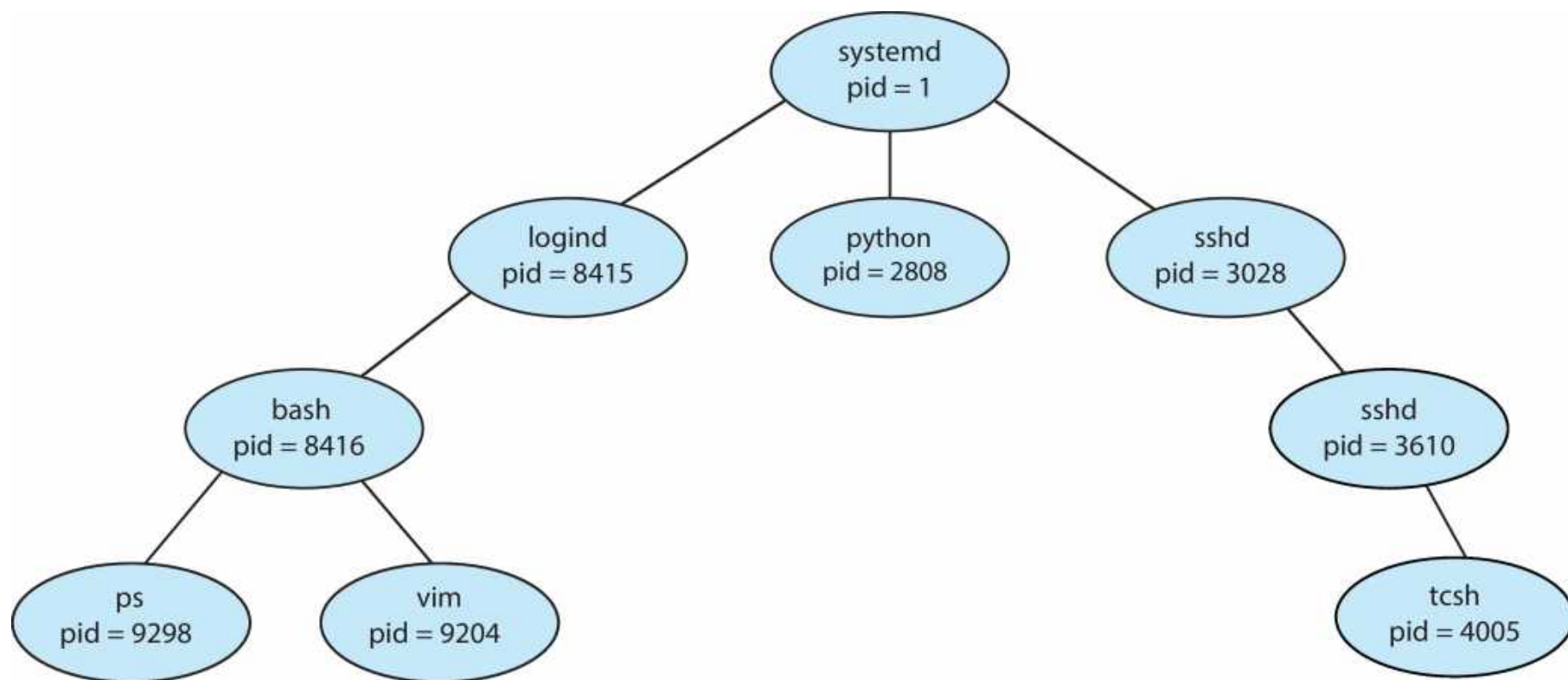
进程创建

- A process may create several new processes
 - **Parent** process: the creating process
 - **Children** processes: the new processes
- The first process
 - The kernel, while it is booting up, creates the first process – **init**.
 - The “**init**” process:
 - has **PID = 1**, and
 - is running the program code “**/sbin/init**”.
 - Its first task is to **create more processes...**



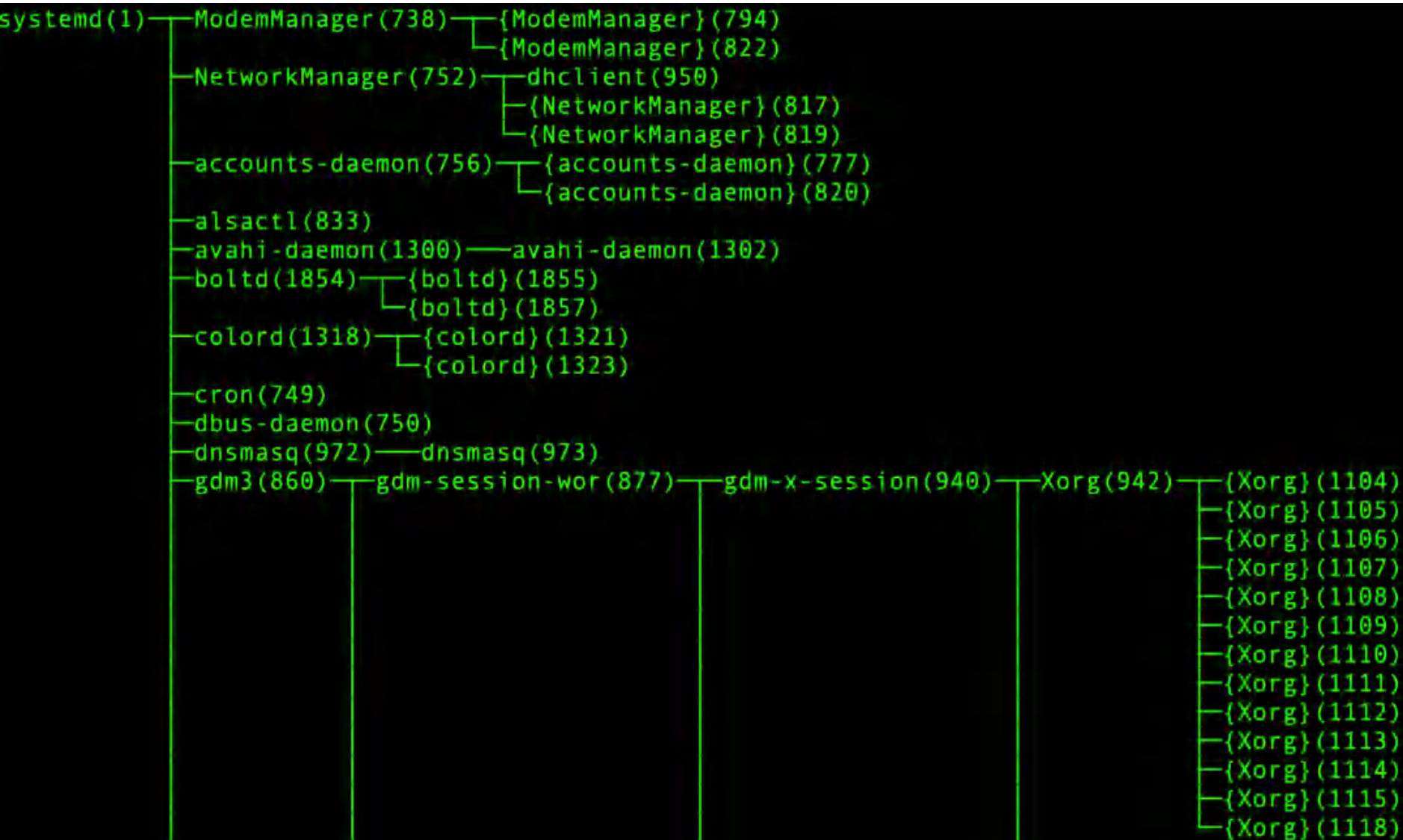
进程操作

➤ 进程的树形结构





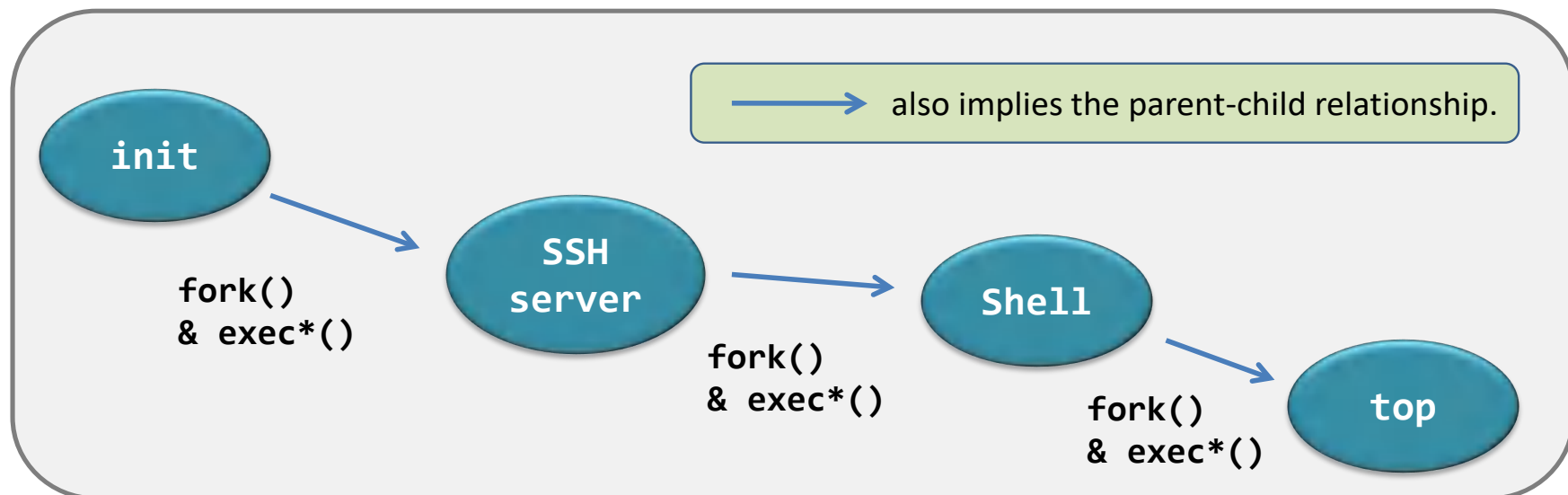
Linux中的进程树





孤儿进程

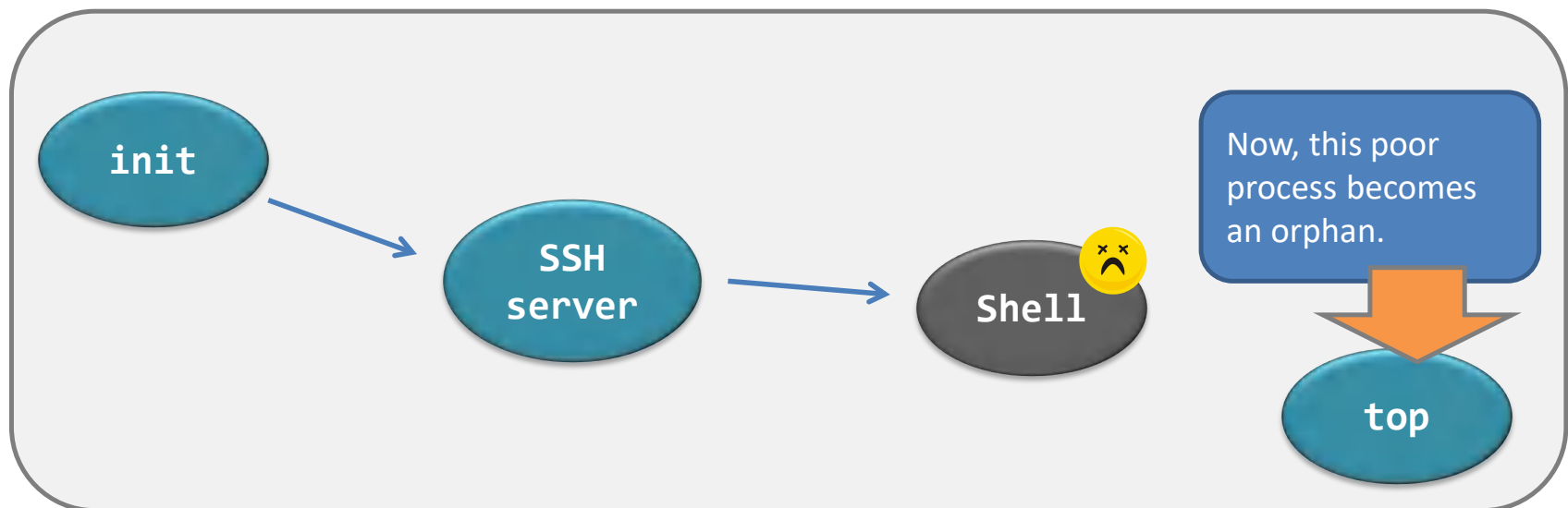
- You can view the tree with the command:
 - “**pstree**”; or
 - “**pstree -A**” for ASCII-character-only display.





孤儿进程

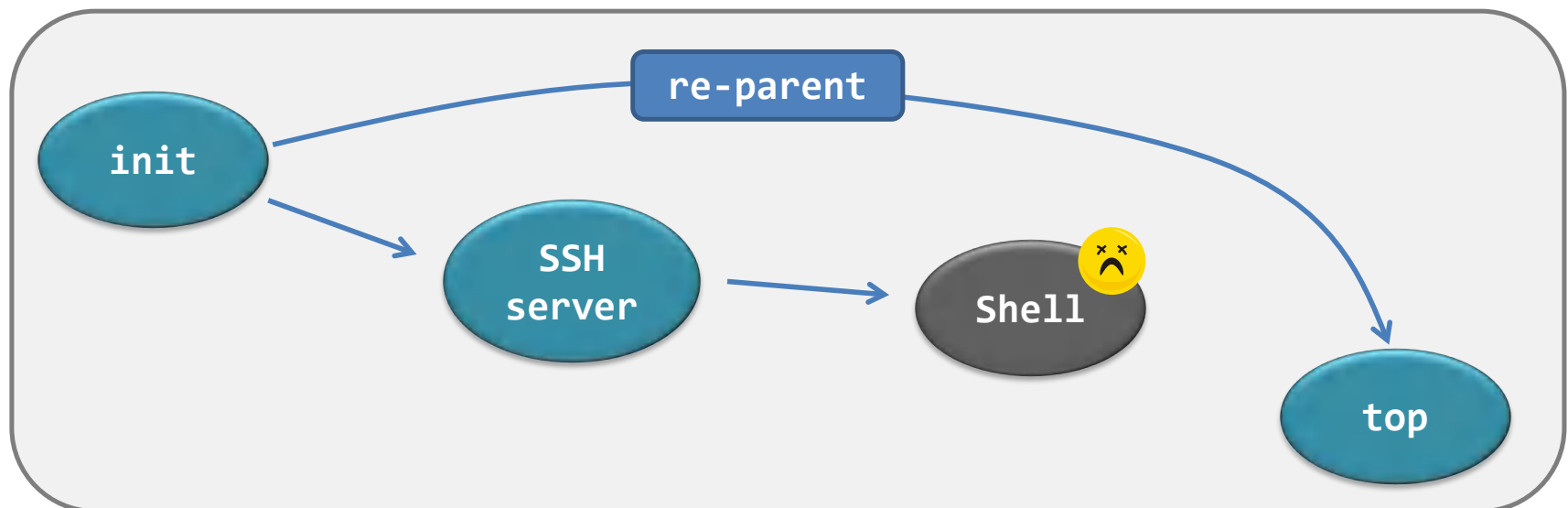
- However, termination can happen, at any time and in any place...
 - All the resources are deallocated to OS when a process terminates
 - A process may become an orphan when its parent terminated
 - An orphan turns the hierarchy from a **tree** into a **forest**!
 - Plus, no one would know the termination of the orphan.





孤儿进程

- In Linux...
 - We have the **re-parent operation**.
 - The “**init**” process will become the step-mother of all orphans.
- Well...Windows maintains a *forest-like* hierarchy.





父子进程之间的关系

- ❖ 父进程创建子进程，子进程又创建其他进程，形成进程树；
- ❖ 通常，通过进程标识符 (pid) 识别和管理过程；
- ❖ 资源共享
 - 父和子共享所有资源
 - 子进程共享父进程资源的子集
 - 父进程和子进程不共享任何资源
- ❖ 执行
 - 父进程和子进程同时并发执行
 - 父进程等待直到子进程终止
- ❖ 地址空间
 - 子进程是父进程的复制品；
 - 子进程加载另一个新的程序；



进程的创建

UNIX示例

- fork () 系统调用创建新进程
- 在fork () 之后使用exec () 系统调用, 用新程序替换进程的内存空间
- 父进程调用wait (), 等待子进程终止

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid;

    /* fork a child process */
    pid = fork();

    if (pid < 0) { /* error occurred */
        fprintf(stderr, "Fork Failed");
        return 1;
    }
    else if (pid == 0) { /* child process */
        execlp("/bin/ls", "ls", NULL);
    }
    else { /* parent process */
        /* parent will wait for the child to complete */
        wait(NULL);
        printf("Child Complete");
    }

    return 0;
}
```



进程创建

- ❖ 进程创建是通过内核系统调用 **fork()** 实现的;
- ❖ 这会导致操作系统在内核模式下执行:

1

- Allocate a slot in the process table for the new process

2

- Assign a unique process ID to the child process

3

- Make a copy of the process image of the parent, with the exception of any shared memory

4

- Increments counters for any files owned by the parent, to reflect that an additional process now also owns those files

5

- Assigns the child process to the Ready to Run state

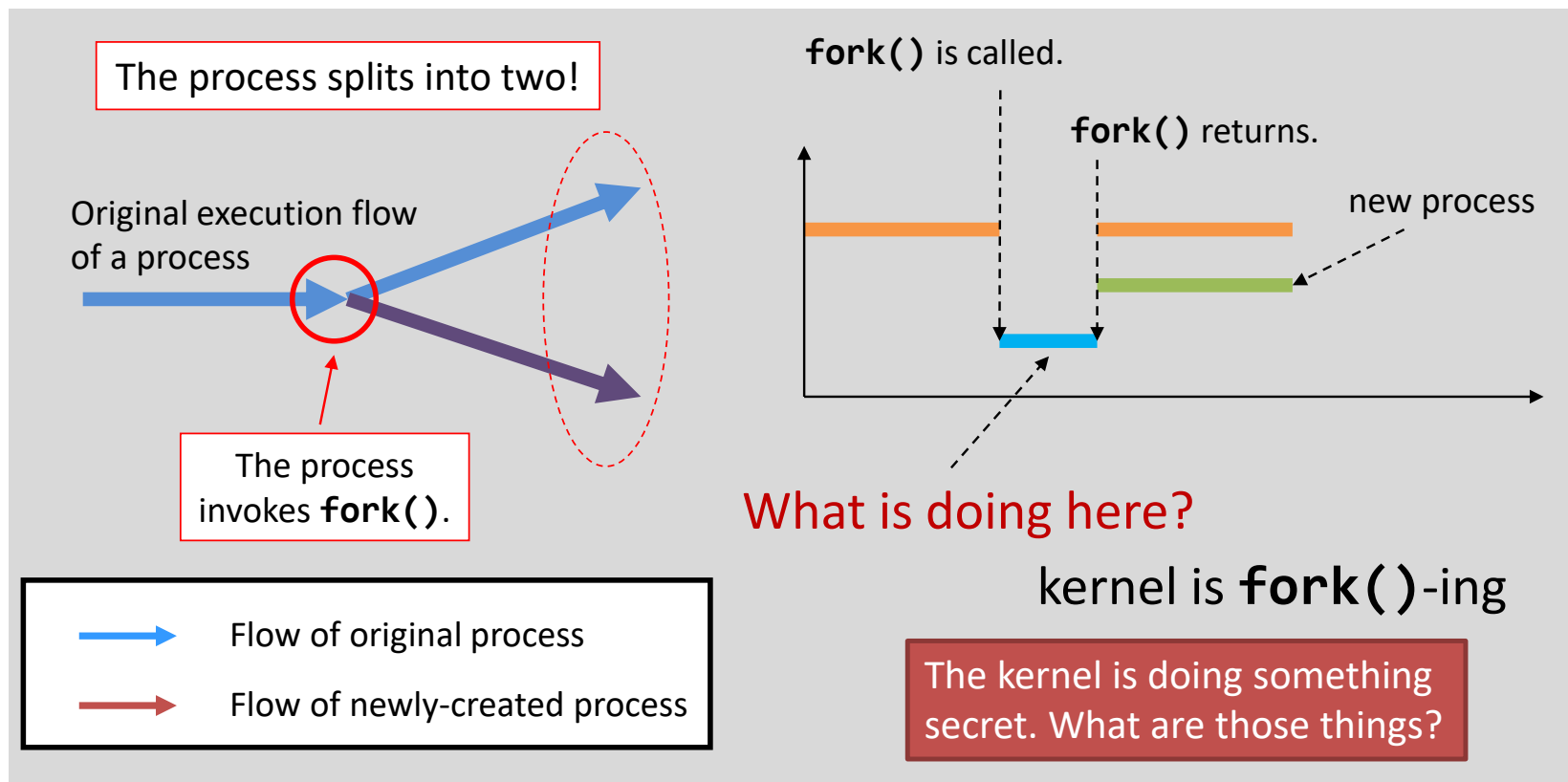
6

- Returns the ID number of the child to the parent process, and a 0 value to the child process



fork()

- From a programmer's view, **fork()** behaves like the following:





进程的创建fork()

- So, how do **fork()** and the processes behave?

```
$ ./fork_example_1  
Ready (PID=1234)  
My PID is 1234  
My PID is 1235  
$_
```

PID 1234

Process 1234 is the original process, and we call it the **parent process**.

PID 1235

```
int main(void) {  
    printf("Ready (PID = %d)\n", getpid());  
    fork();  
    printf("My PID is %d\n", getpid() );  
    return 0;  
}
```

Process 1235 is created by the **fork()** system call, and we call it the **child process**.



进程的创建fork()

- So, how do **fork()** and the processes behave?

```
int main(void) {  
    printf("Ready (PID = %d)\n", getpid());  
    fork();  
    printf("My PID is %d\n", getpid() );  
    return 0;  
}
```

What do we know so far?

- Both the parent and the child execute **the same program before and after fork()**.
- The child process starts its execution **at the location that fork() is returned**, *not from the beginning of the program*.



进程的创建fork()

One more example



```
1 int main(void) {  
2     int result;  
3     printf("before fork ...\n");  
4     result = fork();  
5     printf("result = %d.\n", result);  
6  
7     if(result == 0) {  
8         printf("I'm the child.\n");  
9         printf("My PID is %d\n", getpid());  
10    }  
11    else {  
12        printf("I'm the parent.\n");  
13        printf("My PID is %d\n", getpid());  
14    }  
15  
16    printf("program terminated.\n");  
17 }
```

```
$ ./fork_example_2  
before fork ...
```

PID 1234



进程的创建fork()

One more example



```
1 int main(void) {  
2     int result;  
3     printf("before fork ...\n");  
4     result = fork();  
5     printf("result = %d.\n", result);  
6  
7     if(result == 0) {  
8         printf("I'm the child.\n");  
9         printf("My PID is %d\n", getpid());  
10    }  
11    else {  
12        printf("I'm the parent.\n");  
13        printf("My PID is %d\n", getpid());  
14    }  
15  
16    printf("program terminated.\n");  
17 }
```

```
$ ./fork_example_2  
before fork ...
```

PID 1234

fork()

PID 1235



进程的创建fork()

Assumption

Let there be only **ONE CPU**. Then...

- Only one process is allowed to be executed at one time.
- However, we can't predict which process will be chosen by the OS.
- By the time, this mechanism is called **process scheduling**.

In this example, we assume that the parent, PID 1234, runs first, after the **fork()** call.



进程的创建fork()



```
1 int main(void) {
2     int result;
3     printf("before fork ...\n");
4     result = fork();
5     printf("result = %d.\n", result);
6
7     if(result == 0) {
8         printf("I'm the child.\n");
9         printf("My PID is %d\n", getpid());
10    }
11    else {
12        printf("I'm the parent.\n");
13        printf("My PID is %d\n", getpid());
14    }
15
16    printf("program terminated.\n");
17 }
```

```
$ ./fork_example_2
before fork ...
result = 1235
```

Important

For parent, the return value of **fork()** is the PID of the created child.

PID 1234
(running)

PID 1235
(waiting)



进程的创建fork()

```
1 int main(void) {  
2     int result;  
3     printf("before fork ...\n");  
4     result = fork();  
5     printf("result = %d.\n", result);  
6  
7     if(result == 0) {  
8         printf("I'm the child.\n");  
9         printf("My PID is %d\n", getpid());  
10    }  
11    else {  
12        printf("I'm the parent.\n");  
13        printf("My PID is %d\n", getpid());  
14    }  
15  
16    printf("program terminated.\n");  
17 }
```

```
$ ./fork_example_2  
before fork ...  
result = 1235  
I'm the parent.  
My PID is 1234  
program terminated.
```

PID 1234
(dead)



PID 1235
(waiting)



进程的创建fork()

```
1  int main(void) {  
2      int result;  
3      printf("before fork ...\n");  
4      result = fork();  
5      printf("result = %d.\n", result);  
6  
7      if(result == 0) {  
8          printf("I'm the child.\n");  
9          printf("My PID is %d\n", getpid());  
10     }  
11     else {  
12         printf("I'm the parent.\n");  
13         printf("My PID is %d\n", getpid());  
14     }  
15  
16     printf("program terminated.\n");  
17 }
```

```
$ ./fork_example_2  
before fork ...  
result = 1235  
I'm the parent.  
My PID is 1234  
program terminated.  
result = 0
```

Important

For child, the return value of `fork()` is 0.

PID 1234
(dead)



PID 1235
(running)



进程的创建fork()

```
1 int main(void) {
2     int result;
3     printf("before fork ...\n");
4     result = fork();
5     printf("result = %d.\n", result);
6
7     if(result == 0) {
8         printf("I'm the child.\n");
9         printf("My PID is %d\n", getpid());
10    }
11    else {
12        printf("I'm the parent.\n");
13        printf("My PID is %d\n", getpid());
14    }
15
16    printf("program terminated.\n");
17 }
```

```
$ ./fork_example_2
before fork ...
result = 1235
I'm the parent.
My PID is 1234
program terminated.
result = 0
I'm the child.
My PID is 1235
program terminated.
$ _
```

PID 1234
(dead)



PID 1235
(dead)





进程的创建fork()

- **fork()** behaves like “*cell division*”.
 - It creates the child process by **cloning** from the parent process, including...

Cloned items	Descriptions
Program code [File & Memory]	They are sharing the same piece of code.
Memory	Including local variables, global variables, and dynamically allocated memory.
Opened files [Kernel's internal]	If the parent has opened a file “A”, then the child will also have file “A” opened automatically.
Program counter [CPU register]	That's why they both execute from the same line of code after fork() returns.



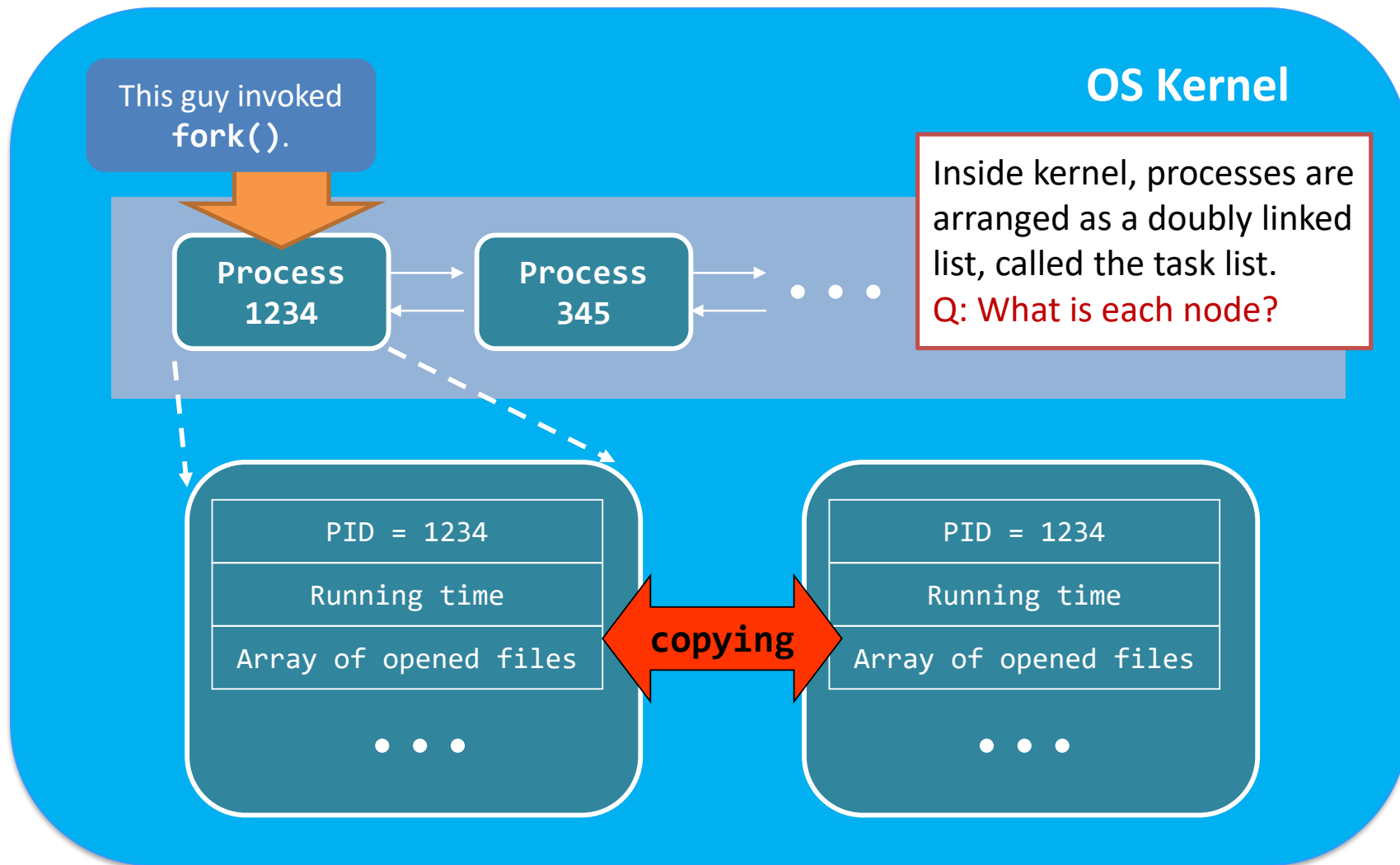
进程的创建fork()

- However...
 - **fork()** does not clone the following...
 - Note: they are all data inside the memory of kernel.

Distinct items	Parent	Child
Return value of fork()	PID of the child process.	0
PID	Unchanged.	Different, not necessarily be "Parent PID + 1"
Parent process	Unchanged.	Doesn't have the same parent as that of the parent process.
Running time	Cumulated.	Just created, so should be 0.

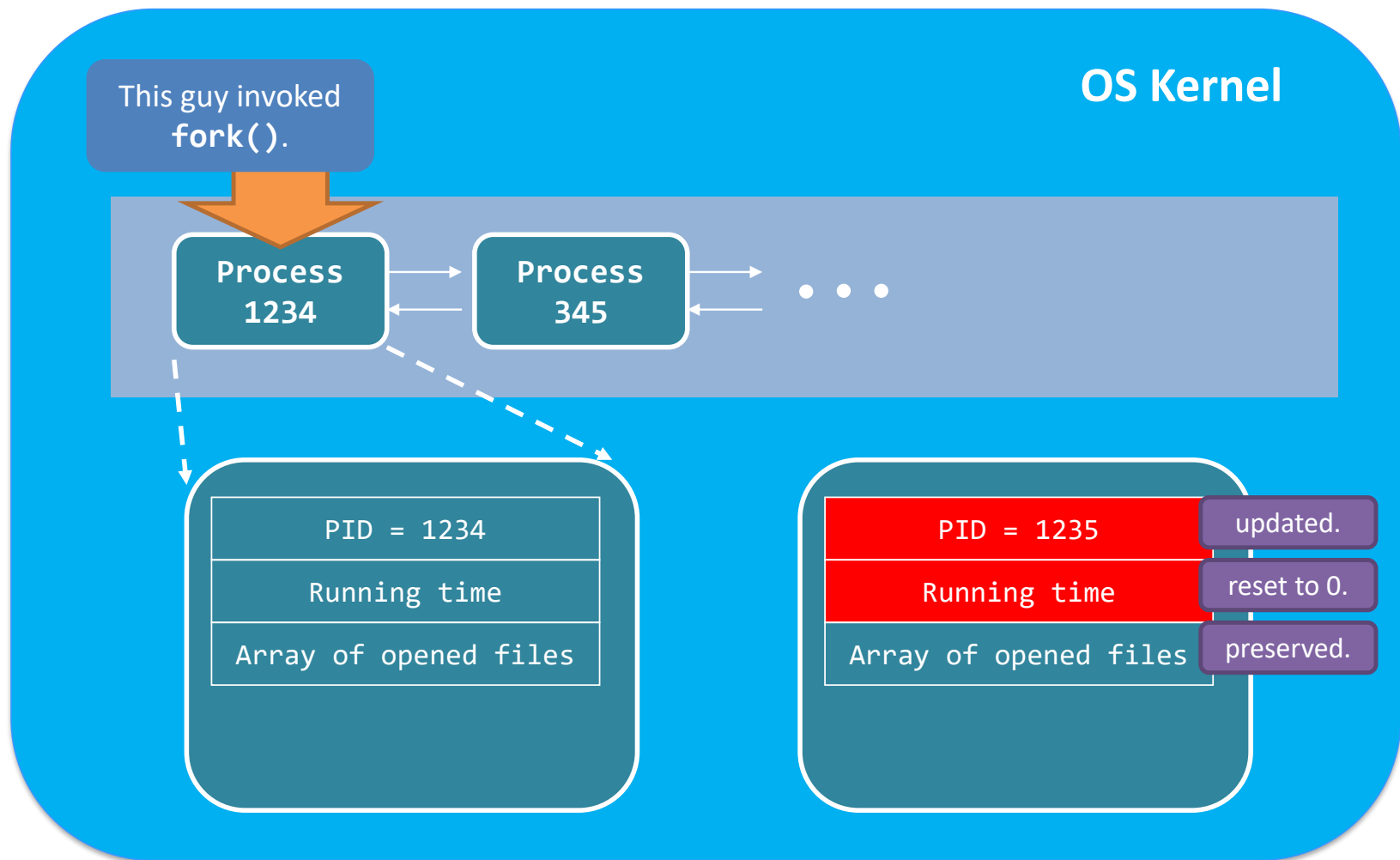


fork() in action



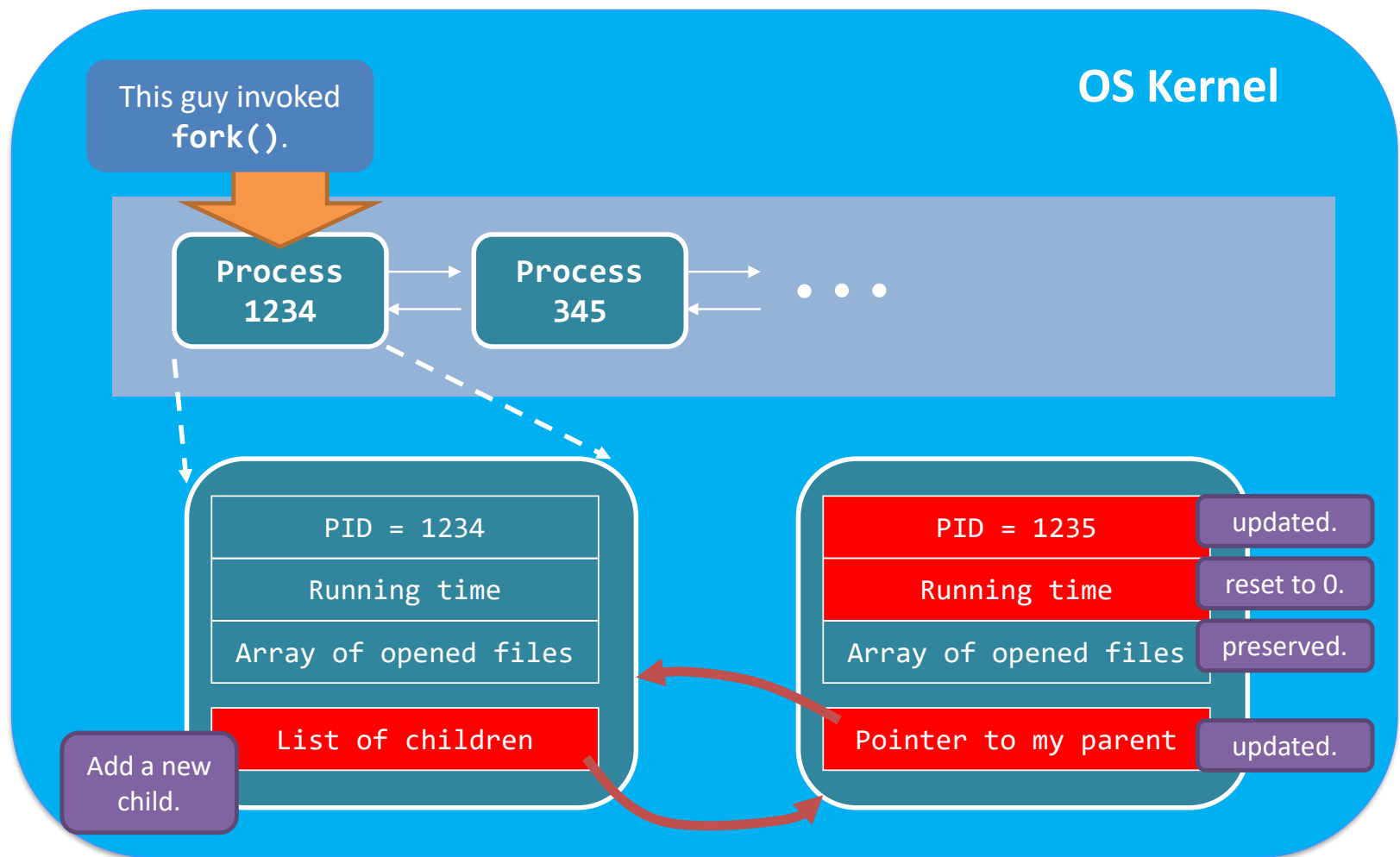


fork() in action – kernel-space update



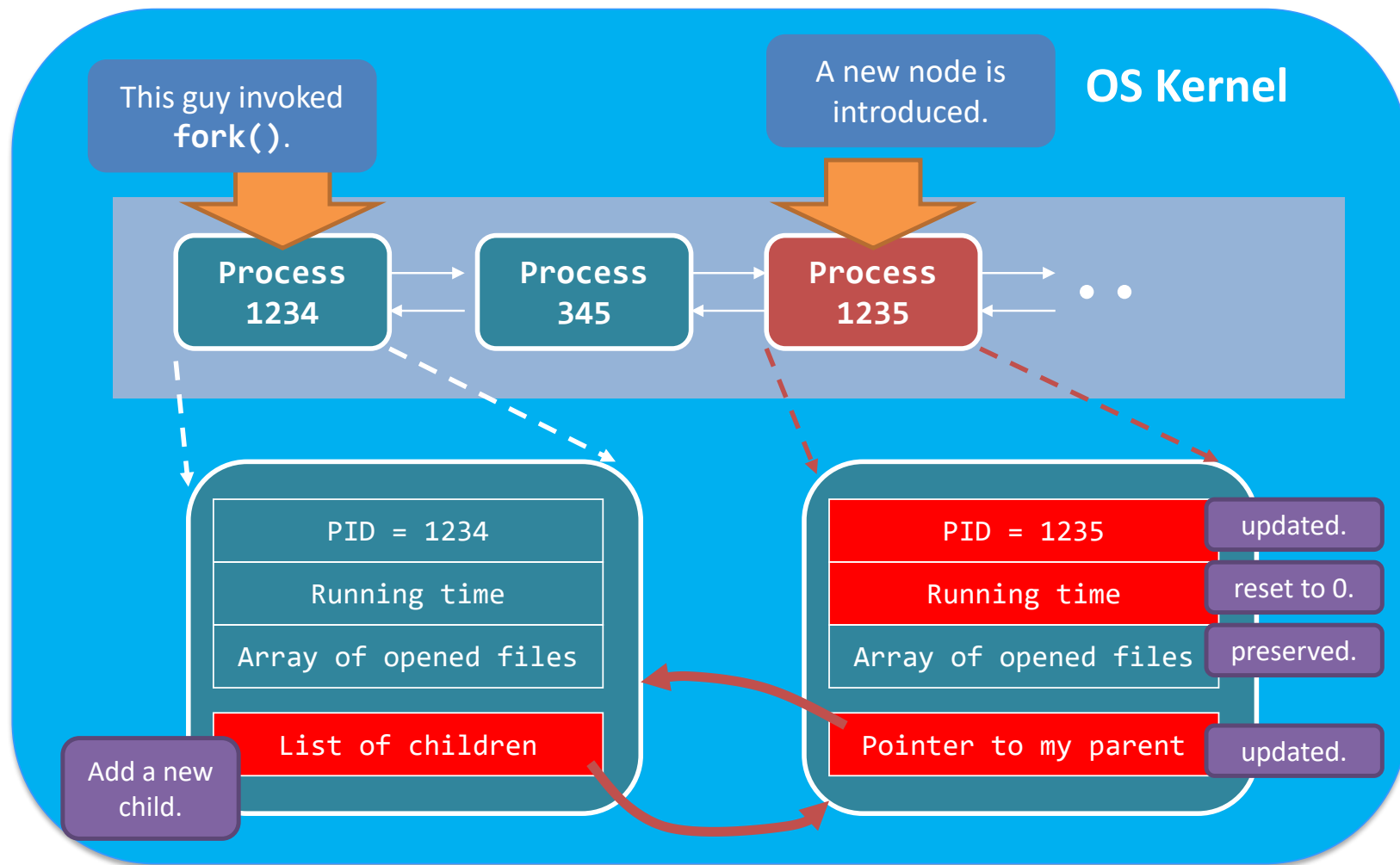


fork() in action – kernel-space update



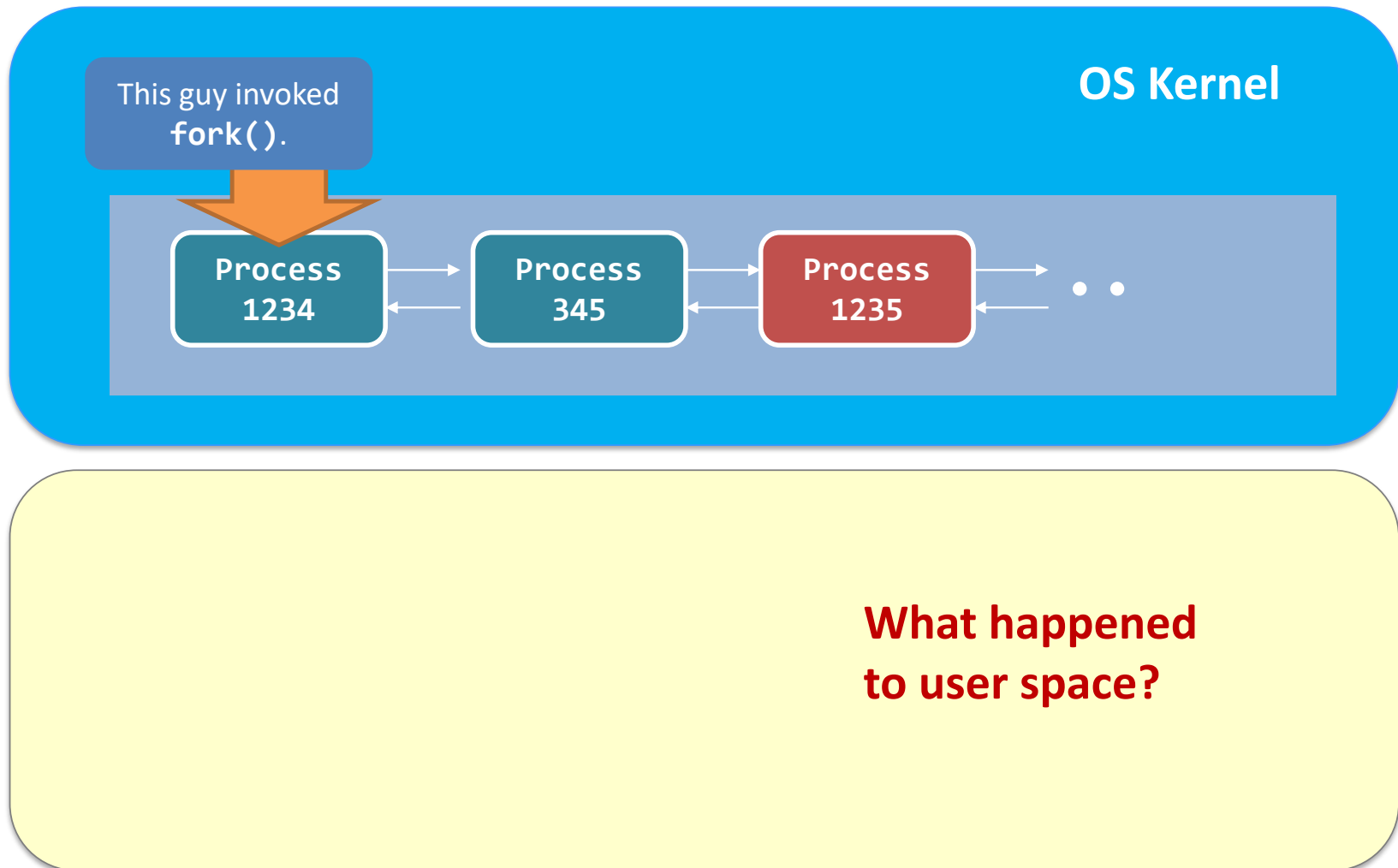


fork() in action – kernel-space update



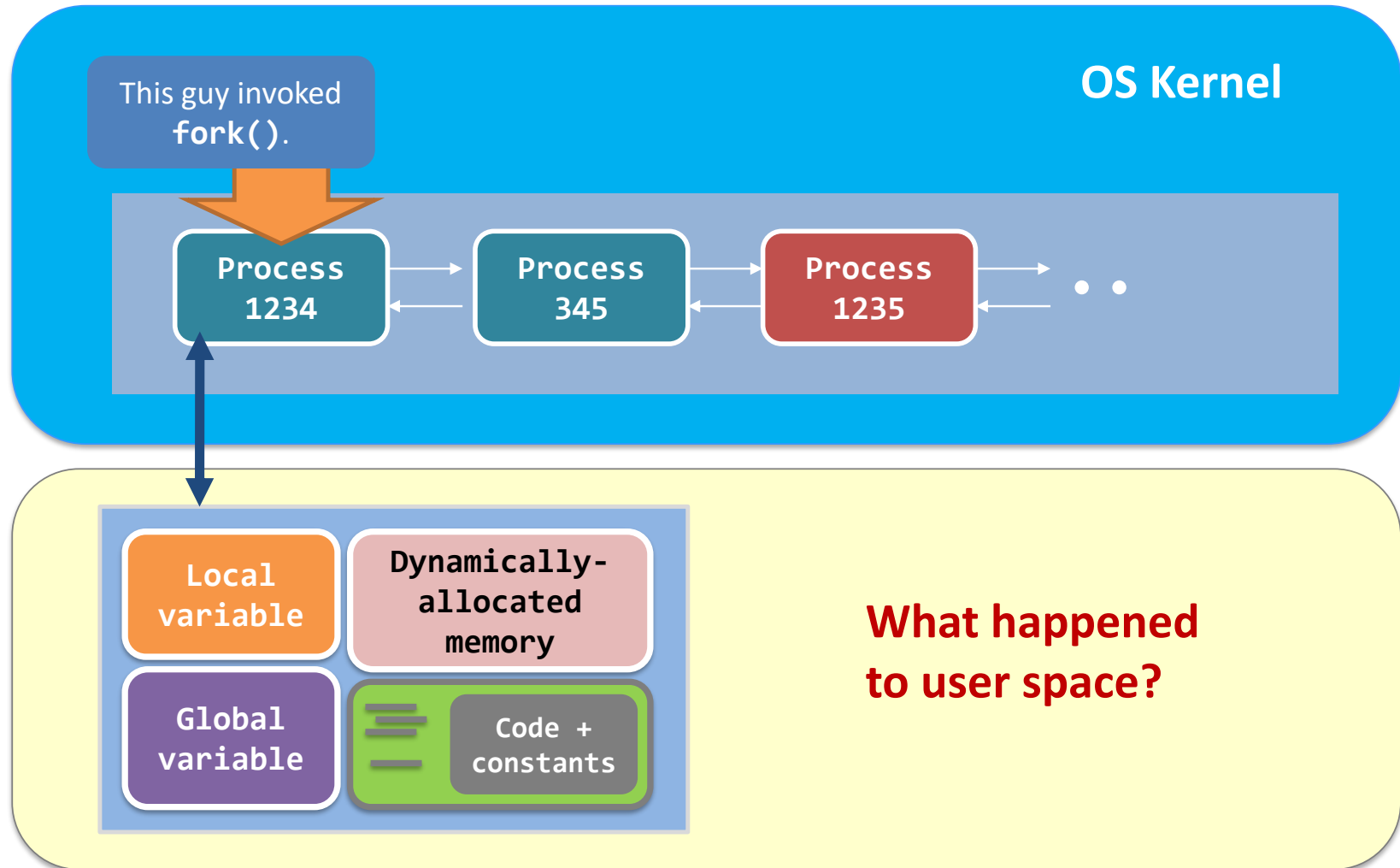


fork() in action – user-space update



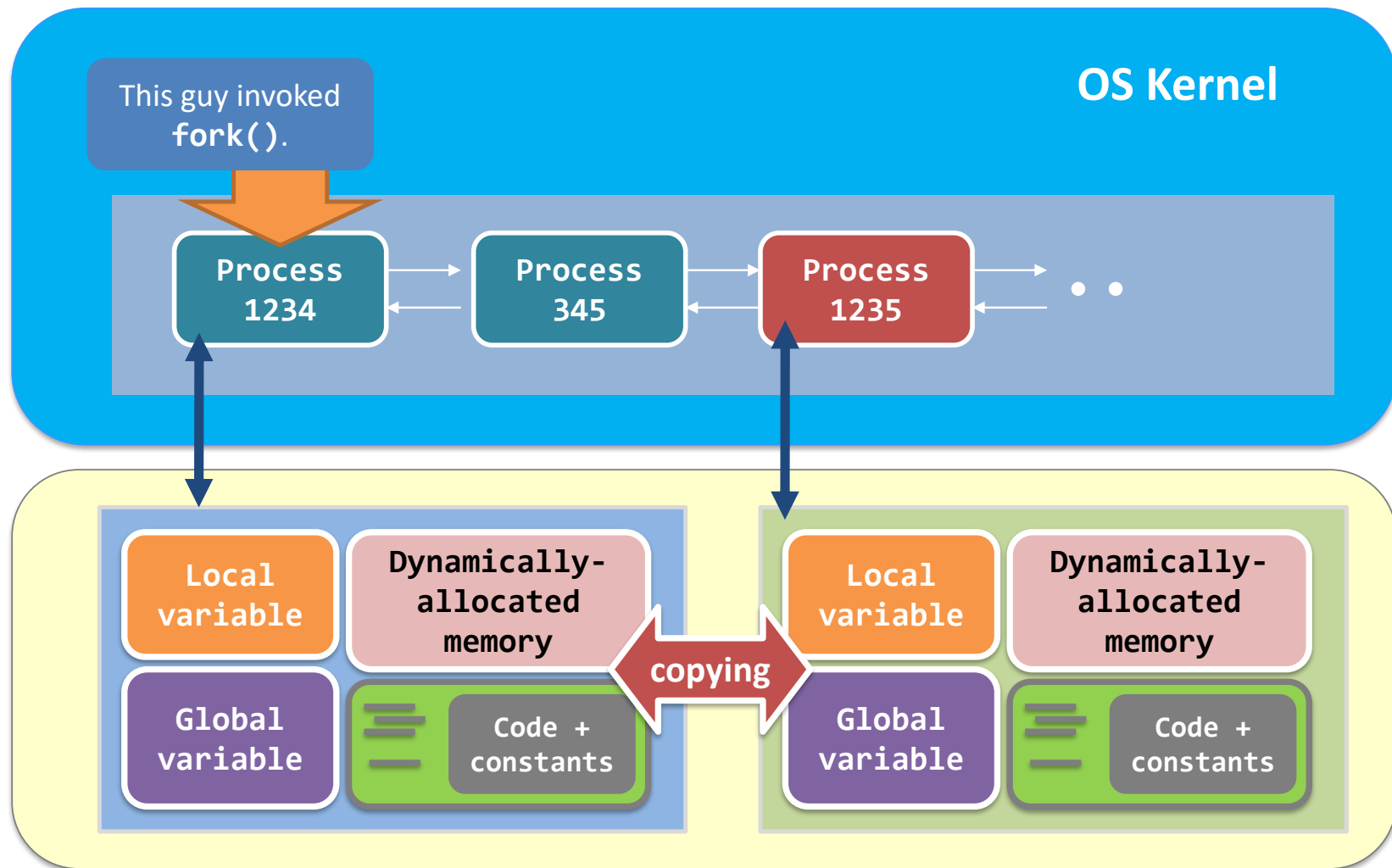


fork() in action – user-space update



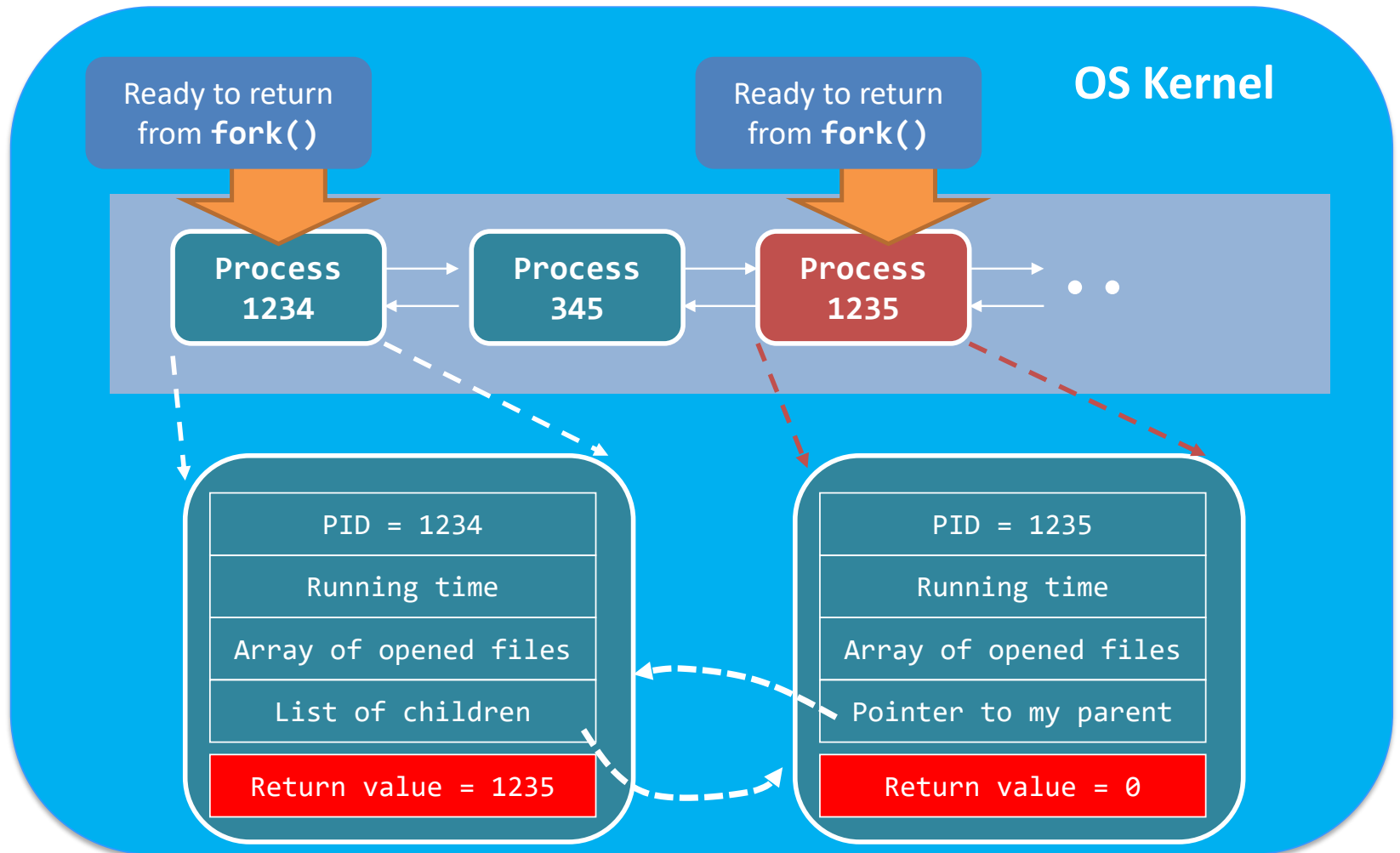


fork() in action – user-space update





fork() in action – finish





fork() 程序示例 (创建一个分离的进程)

```
1 #include <stdio.h>
2 #include <stdlib.h> // exit
3 #include <unistd.h> // fork
4
5 int main() {
6     int x = 100;
7     int rc = fork();
8     if (rc < 0) {
9         // fork failed; exit
10        fprintf(stderr, "fork failed\n");
11        exit(EXIT_FAILURE);
12    } else if (rc == 0) {
13        x = 101;
14        printf("x in child process: %d, the address is %d\n", x, &x);
15    } else {
16        x = 102;
17        printf("x in parent process: %d\n, the address is %d\n", x, &x);
18    }
19    return 0;
20 }
```

Demo



vfork() 程序示例 (创建一个共享的进程)

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <sys/types.h>
4 #include <unistd.h>
5 #include <sys/wait.h>
6
7 int main(void)
8 {
9     int count = 1;
10    pid_t childpid;
11
12    childpid = vfork(); /* child shares parent's address space */
13    if (childpid < 0) {
14        perror("vfork()");
15        return EXIT_FAILURE;
16    }
17    else /* vfork() returns 2 values: 0 for child pro and childpid for parent pro
18        if (childpid == 0) { /* This is child pro, parent hung up until child exit
19            count++;
20            printf("Child pro pid = %d, count = %d (addr = %p)\n", getpid(), count
count);
21            printf("Child taking a nap ...\n");
22            sleep(10);
23            printf("Child waking up!\n");
24            _exit(0); /* or exec(0); "return" will cause stack smashing */
25        }
26        else { /* This is parent pro, start when the vforked child terminated */
```



通过Windows API 创建单独的进程

```
#include <stdio.h>
#include <windows.h>

int main(VOID)
{
    STARTUPINFO si;
    PROCESS_INFORMATION pi;

    /* allocate memory */
    ZeroMemory(&si, sizeof(si));
    si.cb = sizeof(si);
    ZeroMemory(&pi, sizeof(pi));

    /* create child process */
    if (!CreateProcess(NULL, /* use command line */
        "C:\\WINDOWS\\system32\\mspaint.exe", /* command */
        NULL, /* don't inherit process handle */
        NULL, /* don't inherit thread handle */
        FALSE, /* disable handle inheritance */
        0, /* no creation flags */
        NULL, /* use parent's environment block */
        NULL, /* use parent's existing directory */
        &si,
        &pi))
    {
        fprintf(stderr, "Create Process Failed");
        return -1;
    }
    /* parent will wait for the child to complete */
    WaitForSingleObject(pi.hProcess, INFINITE);
    printf("Child Complete");

    /* close handles */
    CloseHandle(pi.hProcess);
    CloseHandle(pi.hThread);
}
```



进程创建之后

- ❖ 创建进程后，内核可以执行以下操作之一，作为调度器例程的一部分：
 - 留在父进程中
 - 将控制权转移到子进程
 - 将控制权转移到另一个进程





程序的执行

- **exec1()** – a member of the **exec** system call family (and the family has 6 members).

```
int main(void) {  
    printf("before exec1 ...\n");  
    exec1("/bin/ls", "/bin/ls", NULL);  
    printf("after exec1 ...\n");  
    return 0;  
}
```

```
$ ./exec_example  
before exec1 ...
```

Arguments of the exec1() call

- 1st argument: the program name, **"/bin/ls"** in the example.
- 2nd argument: 1st argument to the program.
- 3rd argument: indicate the end of the list of arguments.



程序的执行

- **exec1()** – a member of the **exec** system call family (and the family has 6 members).

```
int main(void) {  
    printf("before exec1 ...\n");  
    → exec1("/bin/ls", "/bin/ls", NULL);  
    printf("after exec1 ...\n");  
    return 0;  
}
```

```
$ ./exec_example  
before exec1 ...  
exec_example  
exec_example.c
```

GUESS:
What happens next?



程序的执行

- `exec1()` – a member of the **exec** system call family (and the family has 6 members).

```
int main(void) {  
  
    printf("before exec1 ...\n");  
  
    exec1("/bin/ls", "/bin/ls", NULL);  
  
    printf("after exec1 ...\n");  
  
    return 0;  
}
```

WHAT?!

The shell prompt appears!

```
$ ./exec_example  
before exec1 ...  
exec_example  
exec_example.c  
$ _
```

The output says:

- (1) The gray code block **is not reached!**
- (2) The process is **terminated!**

WHY IS THAT?!



程序的执行

- The **exec** system call family is not simply a function that “invokes” a command.

```
int main(void) {  
    printf("before execl ...\n");  
    execl("/bin/ls", "/bin/ls", NULL);  
    printf("after execl ...\n");  
    return 0;  
}
```

Originally, the process is executing the program “**exec_example**”.





程序的执行

- The **exec** system call family is not simply a function that “invokes” a command.

```
int main(void) {  
    printf("before execl ...\n");  
    execl("/bin/ls", "/bin/ls", NULL);  
    printf("after execl ...\n");  
    return 0;  
}
```

The `execl()` call changes the execution from “`exec_example`” to “`/bin/ls`”

```
/* The program “ls” */  
int main(int argc, char ** argv)  
{  
    .....  
    exit(0);  
}
```





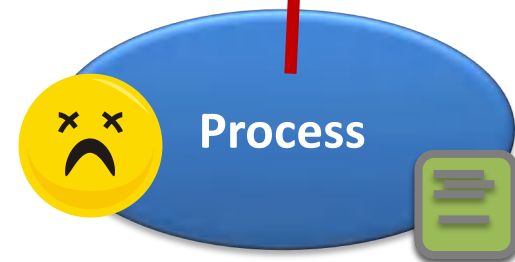
程序的执行

- The **exec** system call family is not simply a function that “invokes” a command.

The “**return**” or the “**exit()**” statement in “**/bin/ls**” will terminate the process...

Therefore, it is certain that the process cannot go back to the old program!

```
/* The program “ls” */  
  
int main(int argc, char ** argv)  
{  
    .....  
    exit(0);  
}
```





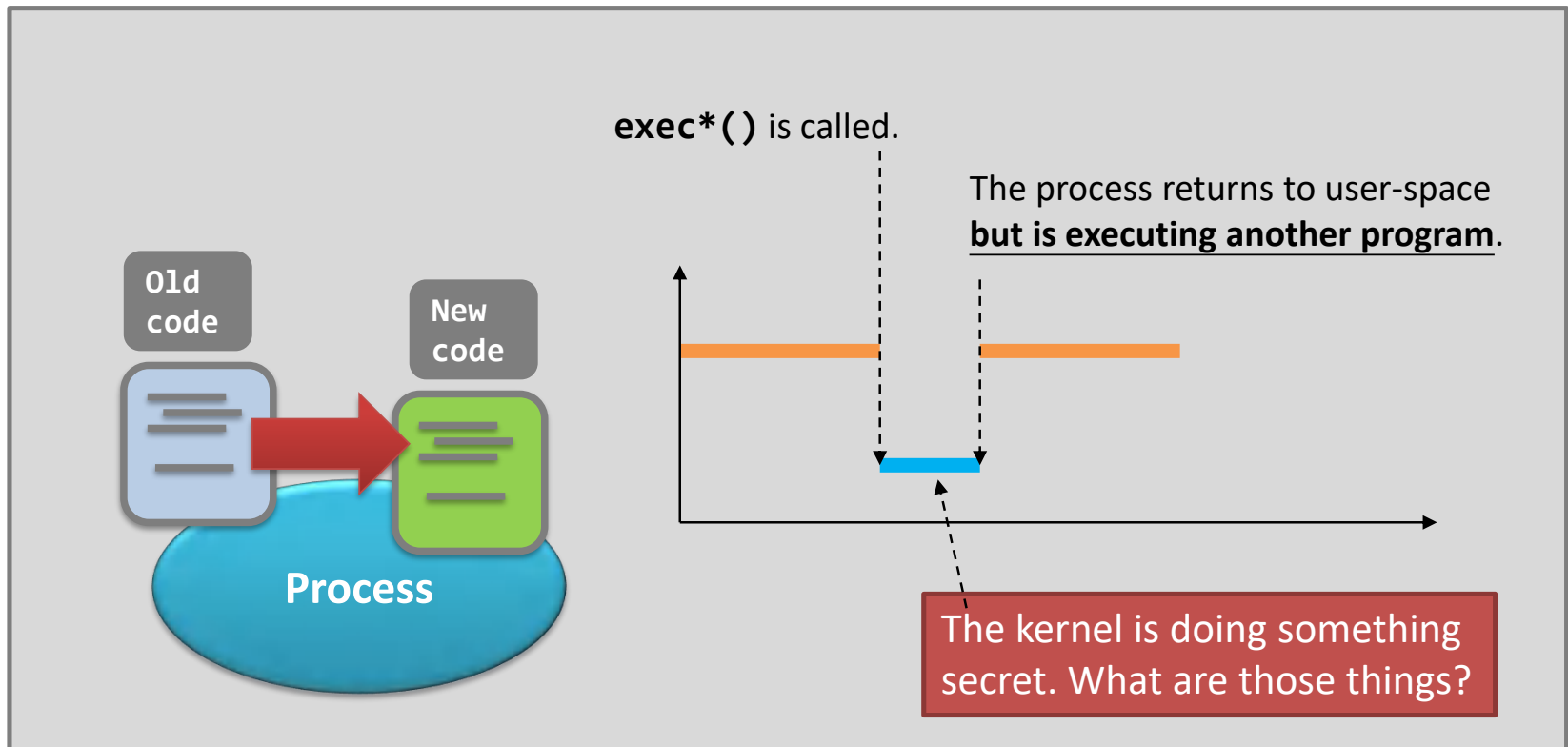
程序的执行

- The process is changing the code that is executing and **never returns to the original code.**
 - The last two lines of codes are therefore not executed.
- The process that calls any one of the member of the exec system call family will **throw away** many things, e.g.,
 - Memory: local variables, global variables, and dynamically allocated memory;
 - Register value: e.g., the program counter;
- But, the process will **preserve** something, including:
 - PID;
 - Process relationship;
 - Running time, etc.



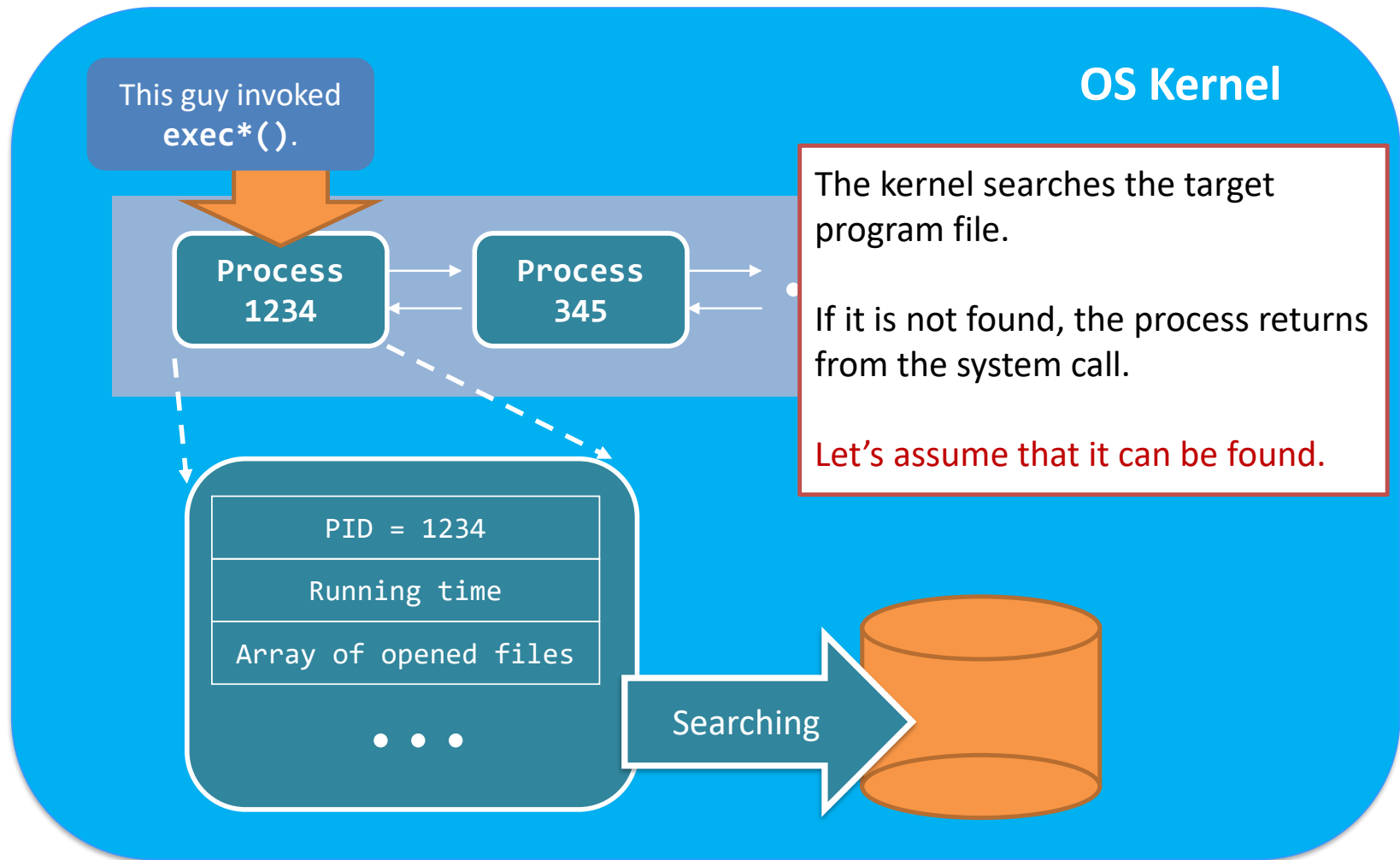
exec*()

- How about the **exec*()** call family?
e.g., `execl("/bin/ls", "/bin/ls", NULL);`



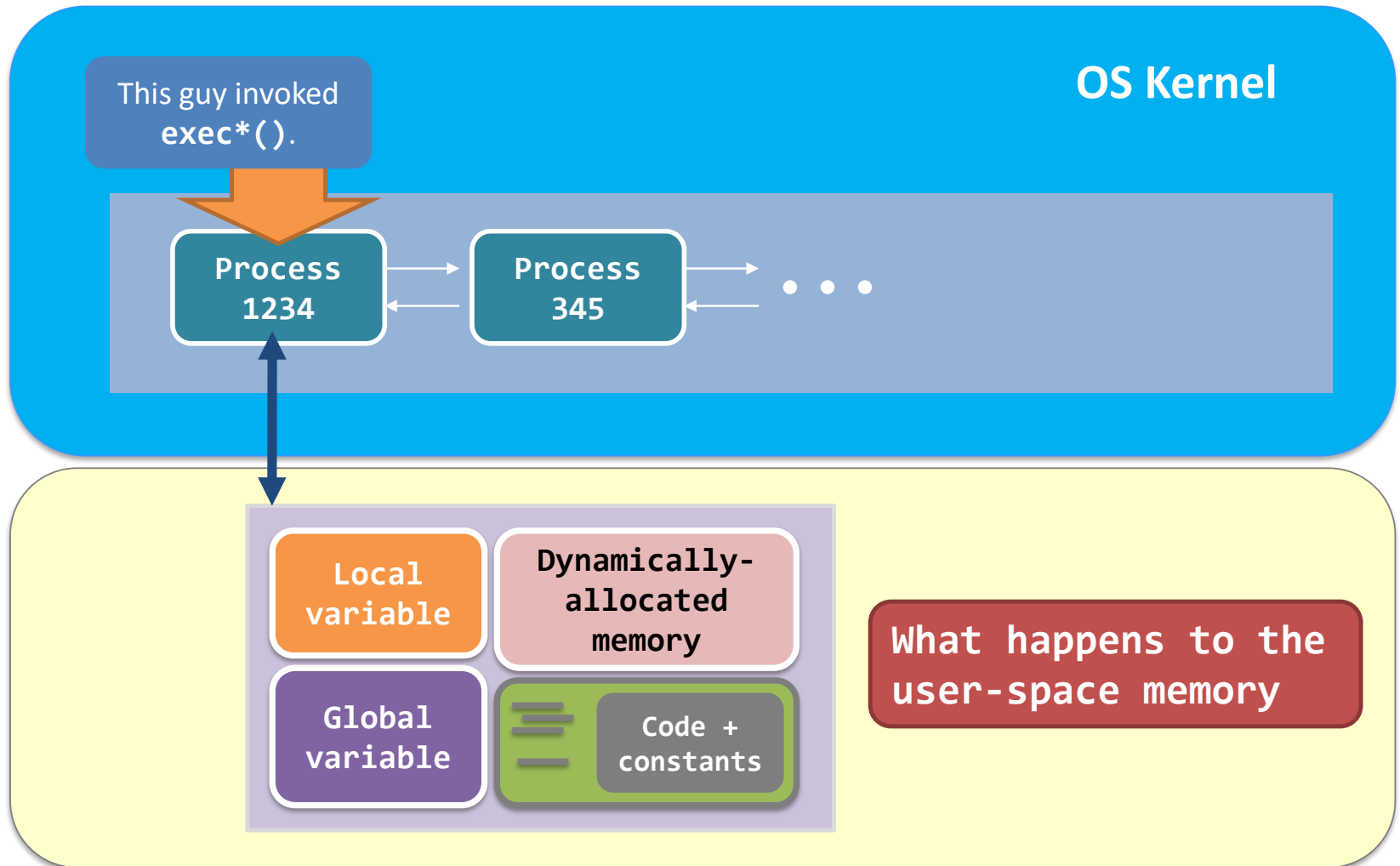


exec*() in action – the start...



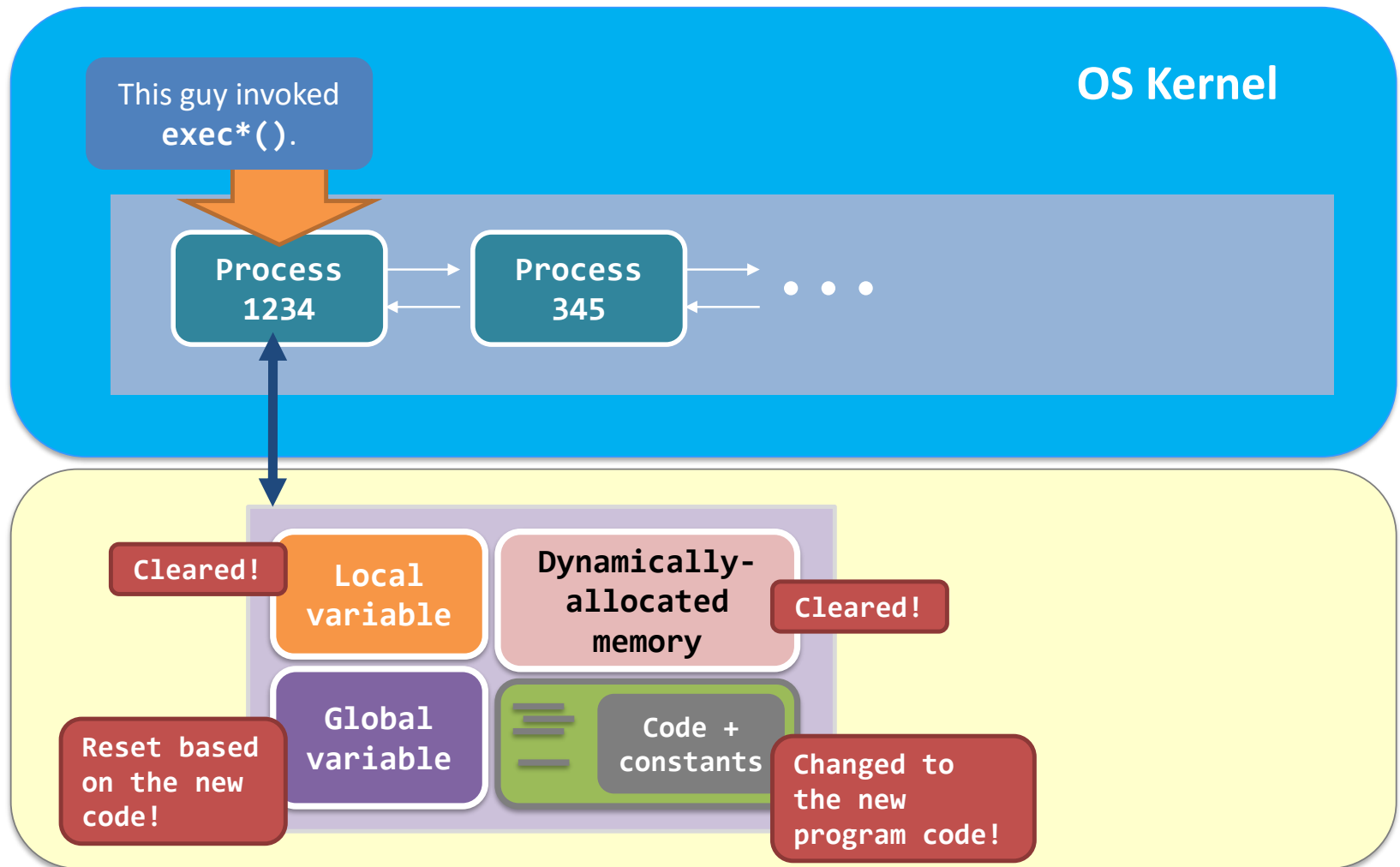


exec*() in action – the end



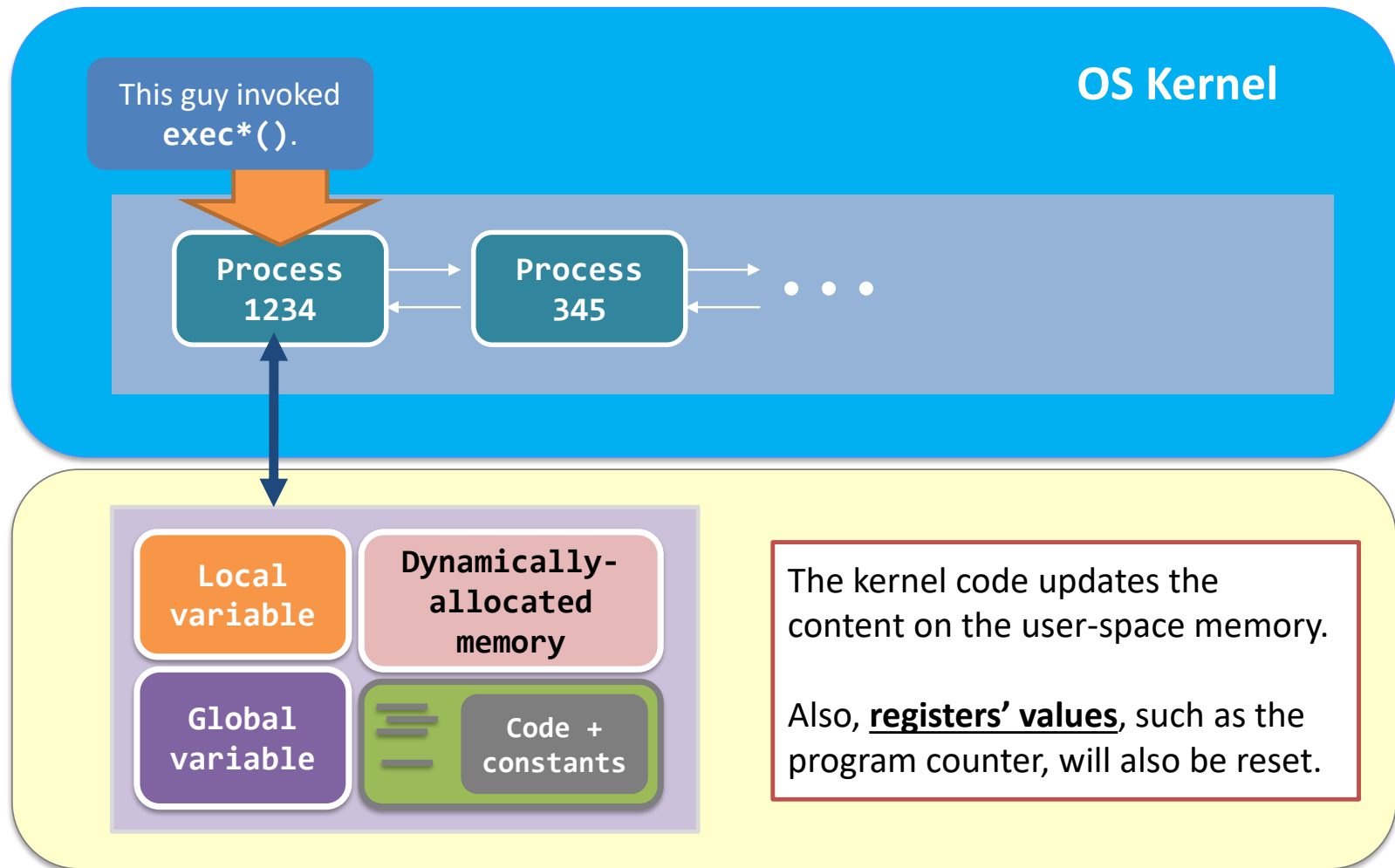


exec*() in action – the end





exec*() in action – the end

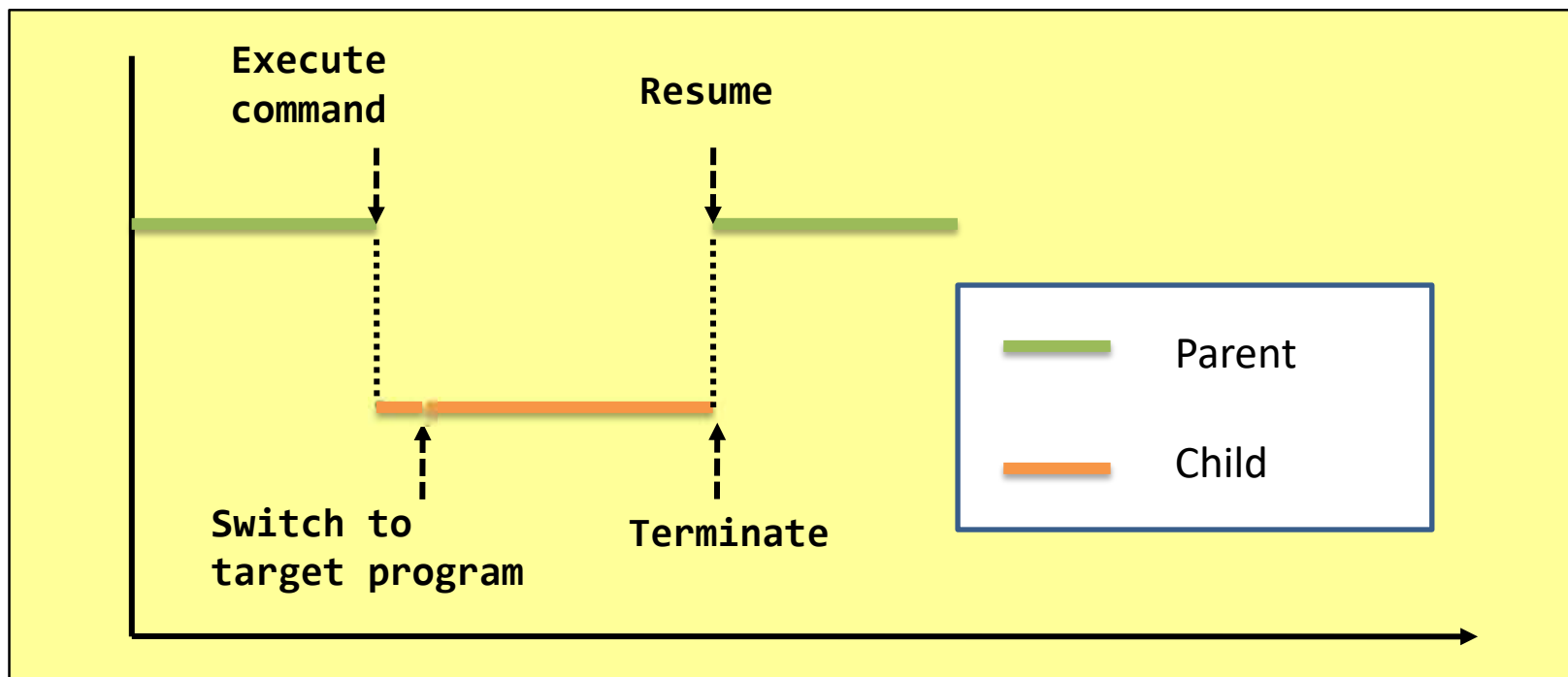




Fork() + exec*()

- The mix can become: 类似于：
 - A shell,
 - The **system()** library call, etc...

Demo





Fork() + exec*()

```
1  int system_test(const char *cmd_str) {
2      if(cmd_str == -1)
3          return -1;
4      if(fork() == 0) {
5          execl(cmd_str, cmd_str, NULL);
6          fprintf(stderr,
7              "%s: command not found\n", cmd_str);
8          exit(-1);
9      }
10     return 0;
11 }
12 int main(void) {
13     printf("before...\n\n");
14     system_test("/bin/ls");
15     printf("\nafter...\n");
16     return 0;
17 }
```

Is this the
only result?

```
$ ./system_implement_1
before...
```

```
system_implement_1
system_implement_1.c
```

```
after...
$ _
```



Fork() + exec*()

```
1  int system_test(const char *cmd_str) {
2      if(cmd_str == -1)
3          return -1;
4      if(fork() == 0) {
5          execl(cmd_str, cmd_str, NULL);
6          fprintf(stderr,
7              "%s: command not found\n", cmd_str);
8          exit(-1);
9      }
10     return 0;
11 }
12 int main(void) {
13     printf("before...\n\n");
14     system_test("/bin/ls");
15     printf("\nafter...\n");
16     return 0;
17 }
```

Some strange cases happened when the program is executed repeatedly!! Why?

```
$ ./system_implement_1
before...
```

```
after...
system_implement_1
system_implement_1.c
$ _
```



Fork() + exec*()

```
1  int system_test(const char *cmd_str) {
2      if(cmd_str == -1)
3          return -1;
4      if(fork() == 0) {
5          execl(cmd_str, cmd_str, NULL);
6          fprintf(stderr,
7              "%s: command not found\n", cmd_str);
8          exit(-1);
9      }
10     return 0;
11 }
12 int main(void) {
13     printf("before...\n\n");
14     system_test("/bin/ls");
15     printf("\nafter...\n");
16     return 0;
17 }
```

Let's re-color the program!

-  Parent process
-  Child process
-  Both processes

```
$ ./system_implement_1
```

```
before...
```

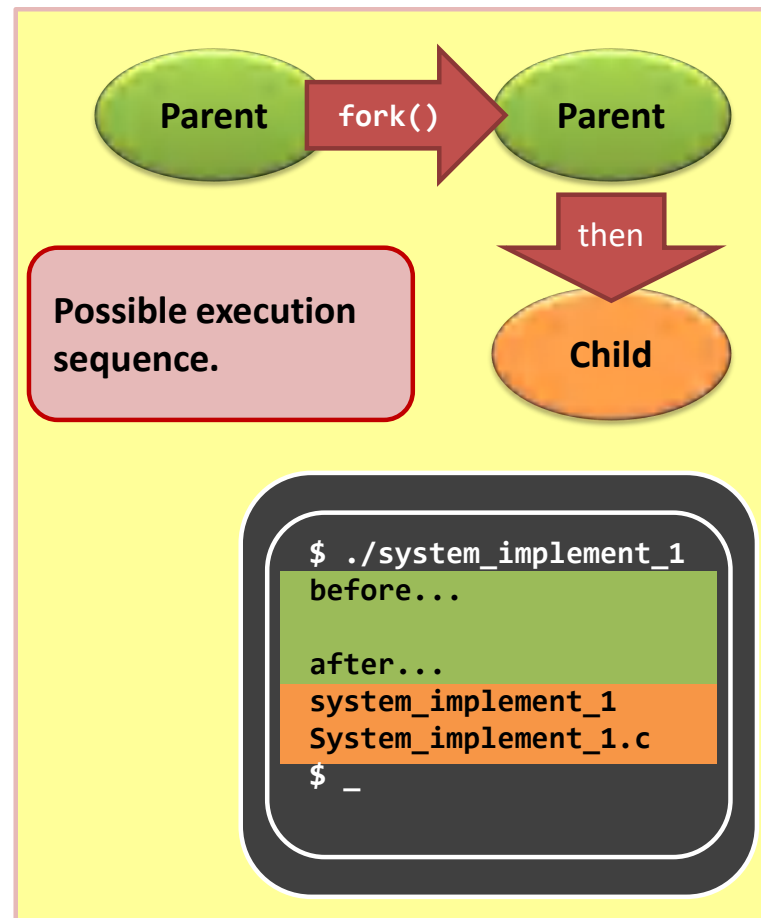
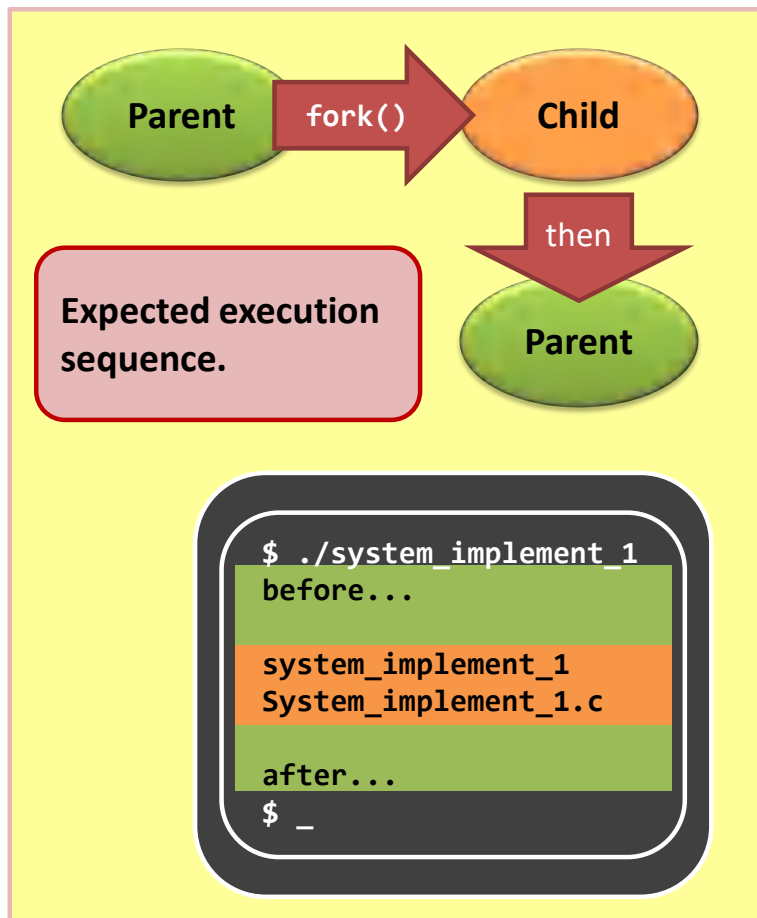
```
after...
```

```
system_implement_1
system_implement_1.c
```

```
$ _
```



Fork() + exec*() != system()





Fork() + exec*() != system()

- How to let the child to execute first?
 - But...we can't control the **process scheduling** of the OS to this extent.
- Then, our problem becomes...
 - How to **suspend** the execution of the parent process?
 - How to **wake** the parent up after the child is terminated?



`fork() + exec*() + wait() = system()`

```
1 int system_test(const char *cmd_str) {
2     if(cmd_str == -1)
3         return -1;
4     if(fork() == 0) {
5         execl("/bin/sh", "/bin/sh",
6             "-c", cmd_str, NULL);
7         fprintf(stderr,
8             "%s: command not found\n", cmd_str);
9         exit(-1);
10    }
11    wait(NULL);
12    return 0;
13 }
14
15 int main(void) {
16     printf("before...\n\n");
17     system_test("/bin/ls");
18     printf("\nafter...\n");
19     return 0;
20 }
```

What is the
output now?



`fork() + exec*() + wait() = system()`

```
1  int system_test(const char *cmd_str) {
2      if(cmd_str == -1)
3          return -1;
4      if(fork() == 0) {
5          execl("/bin/sh", "/bin/sh",
6              "-c", cmd_str, NULL);
7          fprintf(stderr,
8              "%s: command not found\n", cmd_str);
9          exit(-1);
10     }
11     wait(NULL);
12     return 0;
13 }
14
15 int main(void) {
16     printf("before...\n\n");
17     system_test("/bin/ls");
18     printf("\nafter...\n");
19     return 0;
20 }
```

The parent is
suspended until
the child
terminates

```
$ ./system_implement_2
before...
```

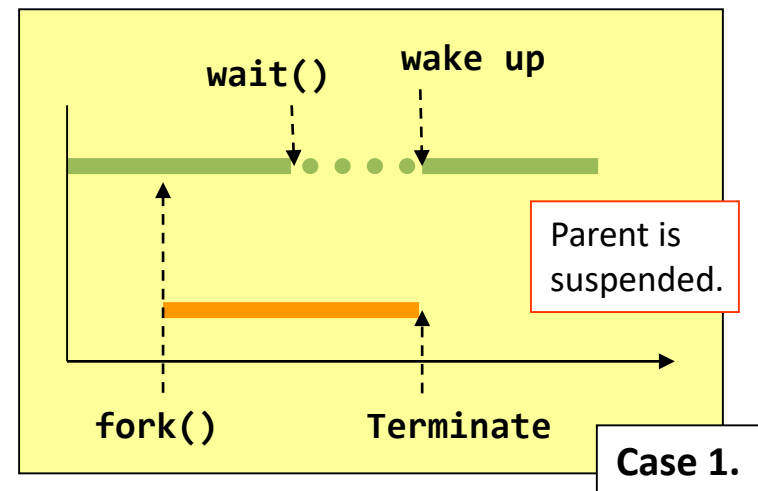
```
system_implement_2
System_implement_2.c
```

```
after...
$ _
```



进程等待wait()

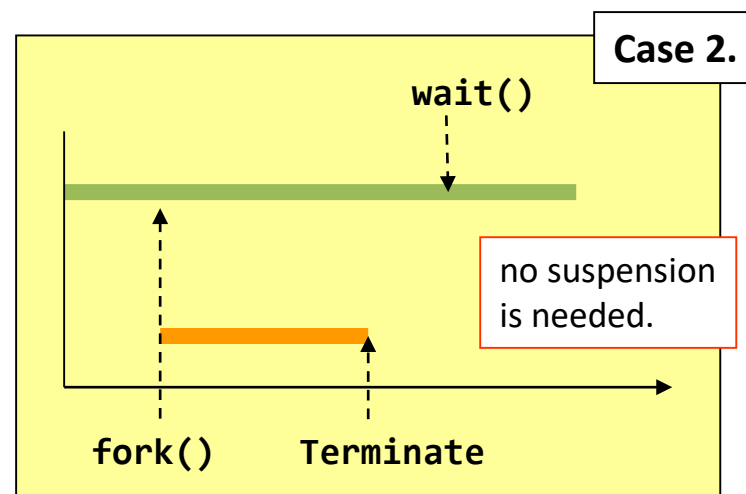
- The **wait()** system call **suspend** the calling parent process (Case 1).
- When to wake up?
 - **wait()** returns and wakes up the calling process when the one of its child processes changes from **running to terminated**.





进程等待wait()

- What happens if
 - There were no running children;
 - There were no children;
- **wait()** does not suspend the calling process (Case 2)





进程等待wait()

- Limitation of **wait()**?
 - waits for any one of the children
 - Detect child termination only
- How to wait for a particular process?
 - **waitpid()**



wait() VS Waitpid()

wait()	waitpid()
Wait for any one of the children.	Depending on the parameters, waitpid() will wait for a particular child only.
Detect child termination only.	Depending on the parameters, waitpid() <u>can detect child's status changing</u> : -from running to suspended, and -from suspended to running.

For more details, you must read the man pages of **wait()** and **waitpid()**.



谢谢